UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Escola Tècnica Superior d'Enginyeria
de Telecomunicació de Barcelona

telecos BCN

Cooperative LiDAR Sensing in Vehicular Scenarios

Bachelor's Degree Thesis
Submitted to the Faculty at the
Escola Tècnica Superior
d'Enginyeria de Telecomunicació de Barcelona
de la Universitat Politècnica de Catalunya
by

Albert Tomàs Ruiz

In partial fulfillment
of the requirements for the

DEGREE IN TELECOMMUNICATION TECHNOLOGIES AND SERVICES ENGINEERING

Advisor: Anna Umbert Juliana

Barcelona, June 2025

ETSETB – Degree in Telecommunication Technologies and Services Engineering

Abstract

This Bachelor's Thesis addresses the blind spots left by single LiDAR sensors in vehicles and mobile robots by introducing a cooperative system in which two robots share and merge their local maps to build a complete 3D view. Individual scans are first aligned to a local map, and the robots' trajectories are estimated over time. A point cloud detector filters out moving objects, leaving only static structures in the final map. The robots' relative positions are calculated from their shared detections, and the cleaned maps are fused into a more complete global model of the environment. This shared information enables faster reactions to dynamic scenes and supports safer navigation.

Resumen

Este Trabajo de Fin de Grado aborda los puntos ciegos que dejan los sensores LiDAR individuales en vehículos y robots móviles mediante un sistema cooperativo en el que dos robots comparten y fusionan sus mapas locales para construir una vista 3D completa. Primero, cada escaneo se alinea con un mapa local y se estiman las trayectorias de los robots a lo largo del tiempo. Un detector en la nube de puntos elimina los objetos en movimiento, dejando solo las estructuras estáticas en el mapa final. A partir de las detecciones compartidas, se calculan las posiciones relativas de ambos robots y los mapas filtrados se fusionan en un modelo global más completo del entorno. Esta información compartida permite reaccionar más rápidamente ante escenas dinámicas y favorece una navegación más segura.

Resum

Aquest Treball de Fi de Grau aborda els punts cecs que deixen els sensors LiDAR individuals en vehicles i robots mòbils mitjançant un sistema cooperatiu en què dos robots comparteixen i fusionen els seus mapes locals per construir una vista 3D completa. Primer, cada escaneig s'alinea amb un mapa local i s'estimen les trajectòries dels robots al llarg del temps. Un detector en el núvol de punts elimina els objectes en moviment, deixant només les estructures estàtiques en el mapa final. A partir de les deteccions compartides, es calculen les posicions relatives de tots dos robots i els mapes filtrats es fusionen en un model global més complet de l'entorn. Aquesta informació compartida permet reaccionar més ràpidament davant escenes dinàmiques i afavoreix una navegació més segura.

Acknowledgements

I would like to express my gratitude to my thesis advisor at UPC, Anna Umbert Juliana, for her guidance, feedback and support throughout this project.

I am equally grateful to Marouan Mizmizi, my tutor at Politecnico di Milano, for welcoming me into the DEIB laboratory and giving me the opportunity to work on this project.

Special thanks to Giovanni Caroli for providing the detection framework I needed, assisting with data capture sessions and guiding me in the software tools used.

I would also like to thank my friends in Milan for all their support and the countless study sessions during my semester abroad.

Finally, I am deeply grateful to my family for their endless support throughout my bachelor's studies.

Revision History and Approval Record

Revision	Date	Author(s)	Description
1.0	26/05/2025	ATR	Document creation
1.1	16/06/2025	ATR, AUJ	Document revision
2.0	19/06/2025	ATR	Revised after review
2.1	22/06/2025	ATR, AUJ	Document revision
3.0	22/06/2025	ATR	Final version

LLISTA DE DISTRIBUCIÓ DEL DOCUMENT

Role	Surname(s) and Name
Student	Tomàs Ruiz, Albert
Project Supervisor 1	Umbert Juliana, Anna
Project Supervisor 2	Mizmizi, Marouan

Written by:		Reviewed and approved by:	
Date	22/06/2025	Date	22/06/2025
Name	Albert Tomàs Ruiz	Name	Anna Umbert Juliana
Role	Project Author	Role	Project Supervisor

Contents

Abstract 2
Acknowledgements 3
Revision History and Approval Record 4
Contents 5
List of Figures 8
List of Tables 10
Abbreviations 11
1. Introduction 12
1.1. Work Goals 12
1.2. Requirements and Specifications 13
1.3. Methods and Procedures 14
1.4. Work Plan 15
1.5. Deviations 17
2. State of the Art 18
2.1. LiDAR Sensing and Hardware 18
2.2. Point Cloud Preprocessing 20
2.3. Registration Techniques 21
2.3.1. Nearest-Neighbour Search: K-D Tree 21
2.3.2. Point Cloud Alignment Methods 22
2.3.3. Feature Extraction: FPFH + RANSAC 23
2.4. LiDAR SLAM for Trajectory Estimation 23
2.5. Object Detection on LiDAR Data 24
2.5.1. YOLO-Based Object Detection 25
2.6. Multi-LiDAR Map Fusion 25
3. Methodology and Project Development 27
3.1. Experimental Setup 27
3.2. Pipeline Overview 28
3.3. Preprocessing, Single-Robot SLAM and Semantic Detection 29

3.3.1. Packet Decoding 30
3.3.2. Noise Filtering and Downsampling 30
3.3.3. YOLO-Based Object Detection 31
3.3.4. Scan-to-Map ICP and Pose Chain 33
3.3.5. Map Update and Map Export 34
3.4. Dual LiDAR Fusion 34
3.4.1. Map Alignment Using Bounding Boxes 34
3.4.2. Rotation Search Using K-D Tree Fitness Score 35
3.4.3. ICP Refinement 35
3.5. Limitations and Mitigation 36
3.5.1. No Dual-Robot Capture 36
3.5.2. YOLO Model 36
3.5.3. Anchor Pedestrian 37
3.5.4. IMU Data 37
4. Results 38
4.1. Experiments and Tests 38
4.1.1. Test Conditions 38
4.1.2. YOLO Detection Performance 39
4.1.3. SLAM Alignment 40
4.1.4. Map Fusion 41
4.2. Data Visualization 42
4.2.1. Semantic Mapping Validation 42
4.2.2. Single-LiDAR SLAM Map Components 43
4.2.3. Frame-by-Frame Metrics 45
4.2.4. Dual-LiDAR Fusion Map 47
4.3. Limitations 48
5. Sustainability Analysis and Ethical Implications 49
5.1. Sustainability Matrix 49
5.1.1. Environmental Impact 49
5.1.2. Economic Impact 52
5.1.3. Social Impact 53
5.2. Ethical Implications 54
5.3. Relationship with the Sustainable Development Goals 54
6. Conclusions and Future Work 55

6.1. Conclusions 55
6.2. Future Directions 56
Bibliography 57
Appendix A. Detailed SLAM and Fusion Results 60
A.1. Test 1 60
A.2. Test 2 62
A.3. Test 4 64

List of Figures

Figure 1.1: Gantt diagram of the project. 17
Figure 2.1: Time of Flight principle used in LiDAR systems. 18
Figure 2.2: 3D point cloud representation of an indoor environment. 19
Figure 2.3: LiDAR sensor overview. 20
Figure 2.4: Example of a CSV file generated by the LiDAR sensor. 20
Figure 2.5: YOLO's object detection pipeline. 25
Figure 3.1: Experimental setup for data acquisition. 28
Figure 3.2: Single-robot SLAM block diagram. 28
Figure 3.3: Dual-robot map fusion block diagram. 29
Figure 3.4: PandarView 2 software showing a frame from a PandarXT-32 PCAP capture in an urban environment. 30
Figure 3.5: Point cloud before and after preprocessing. 31
Figure 3.6: Bird's-eye view images from an indoor scan. 32
Figure 3.7: Matched 3D points for detected objects. 33
Figure 3.8: Fusion based on pedestrian detection. The second map is translated so that the detected pedestrian (green) aligns in both point clouds, with LiDAR 1 shown in red and LiDAR 2 in blue. 35
Figure 4.1: Bird's-eye views illustrating how pedestrian orientation affects detection performance. 39
Figure 4.2: Raw LiDAR point cloud views with three pedestrians present. 42
Figure 4.3: BEV image generation before and after applying the detector. 42
Figure 4.4: Detected pedestrians highlighted on the static point cloud. 43
Figure 4.5: Single-robot SLAM results for Test 3 using the first 400 frames. 44
Figure 4.6: Static map and pedestrian detections for Test 3. 44
Figure 4.7: Top view of the single-robot SLAM map for Test 3 after 550 frames. 45
Figure 4.8: Inlier RMSE and fitness score per frame for Test 3 (1,737 frames). 46
Figure 4.9: Point cloud statistics per frame for Test 3 (1,737 frames). 47

Figure 4.10: Comparison of the first 250 frames of the original and fused static maps for Test 3. 48

Figure A.1: Inlier RMSE and fitness score per frame for Test 1 (5,016 frames). 60

Figure A.2: Point cloud statistics per frame for Test 1 (5,016 frames). 61

Figure A.3: Static map and trajectory for Test 1 comparing with and without displayed pedestrian detections. 61

Figure A.4: Comparison of fused and single-sensor static maps for Test 1 (first 250 frames) 62

Figure A.5: Inlier RMSE and fitness score per frame for Test 2 (1,256 frames). 62

Figure A.6: Point cloud statistics per frame for Test 2 (1,256 frames). 63

Figure A.7: Static map and trajectory for Test 2 comparing with and without displayed pedestrian detections. 63

Figure A.8: Comparison of fused and single-sensor static maps for Test 2 (first 250 frames) 64

Figure A.9: Inlier RMSE and fitness score per frame for Test 4 (3,434 frames). 64

Figure A.10: Point cloud statistics per frame for Test 4 (3,434 frames). 65

Figure A.11: Static map and trajectory for Test 4 comparing with and without displayed pedestrian detections. 65

Figure A.12: Comparison of fused and single-sensor static maps for Test 4 (first 250 frames) 66

List of Tables

Table 1.1: Work Packages (WPs), milestones, and delivery dates..... 16
Table 2.1: Key specifications of the HESAI PandarXT-32 LiDAR sensor [7]..... 19
Table 4.1: Test runs used for SLAM and fusion evaluation..... 39
Table 4.2: ICP alignment statistics for Test 3 (averaged every 100 frames)..... 40
Table 4.3: Accuracy metrics for the dual-LiDAR fusion simulation..... 41
Table 5.1: Sustainability matrix. 49
Table 5.2: Estimated CO2eq emissions from email exchanges..... 50
Table 5.3: Estimated CO_2eq emissions from the four online meetings..... 50
Table 5.4: Total estimated CO_2eq emissions from all sources..... 51
Table 5.5: Estimated development, deployment and annual maintenance costs..... 53

Abbreviations

API	Application Programming Interface
BEV	Bird's-eye View
CSV	Comma-Separated Values
DEIB	Dipartimento di Elettronica, Informazione e Bioingegneria
ETSETB	Escola Tècnica d'Enginyeria de Telecomunicació de Barcelona
GICP	Generalized Iterative Closest Point
ICP	Iterative Closest Point
IMU	Inertial Measurement Unit
K-D Tree	K-Dimensional Tree
LiDAR	Light Detection and Ranging
NDT	Normal Distributions Transform
PCAP	Packet Capture
PLY	Polygon File Format
Polimi	Politecnico di Milano
RMSE	Root-Mean-Square Error
SLAM	Simultaneous Localization and Mapping
YOLO	You Only Look Once

CHAPTER 1

Introduction

Autonomous vehicles and mobile robots increasingly rely on LiDAR sensing to perceive their surroundings. However, a single scanner leaves blind zones and shadows caused by walls, cars or people who walk into the view, which compromises mapping quality and therefore endangers both pedestrians and drivers. By sharing and merging point clouds from multiple LiDARs, these individual views become a more complete and reliable model of the environment, especially in busy areas filled with moving cars and pedestrians, making navigation clearer and safer.

1.1. Work Goals

This thesis was carried out during an Erasmus exchange at Politecnico di Milano (Leonardo campus, DEIB group). Its purpose is to build an algorithm that allows two LiDAR-equipped robots to share their scans and fuse them into one accurate 3D map of the surroundings.

The practical scenario is as follows: a lead robot moves first, while a second one follows a few metres behind. Since they both share a complete map, when the front robot detects a pedestrian or a moving car, the follower receives this information early and can slow down or adjust the route more quickly than it could on its own.

To achieve this outcome, the project has the following goals:

1. **Align each incoming scan**

   Register every LiDAR frame to the robot's current local map using ICP [1].

2. **Estimate the robot trajectory (LiDAR SLAM)**

   Accumulate the frame-to-map transformations to reconstruct the path followed by each robot [2].

3. **Detect people and cars**

   Apply a YOLO-based network [3] on a bird's-eye projection of the points to place 2D bounding boxes around the objects.

4. **Filter dynamic objects from the static map**

Remove points corresponding to moving objects so the final map contains only the structures and static objects.

5. **Estimate the relative pose between the robots**

   Use shared detections to initialise the transformation linking the two individual maps.

6. **Fuse both static maps into one**

   Merge the aligned clouds to build a more complete global map that both robots can use for safer navigation.

1.2. Requirements and Specifications

The project requirements are as follows:

1. **Preprocess LiDAR streams**

   Use the PandarView 2 [4] software to convert the raw packets (from a PCAP file) from the two HESAI PandarXT-32 sensors to XYZ point cloud CSV files.

2. **Convert YOLO detections to 3D clusters**

   Transform each 2D bounding box (person, car) into its matching 3D cluster inside the cloud.

3. **Filter dynamic objects**

   Build a cloud containing all detected clusters and remove them from the static map.

4. **Synchronise timestamps**

   Synchronise the two LiDAR captures in time and bring both scans into the same coordinate frame.

5. **Fuse the static maps**

   Merge the two cleaned clouds into a single, more complete 3D map.

6. **Track and display robot trajectories**

   Accumulate frame-to-map transformations (LiDAR SLAM) to show the path in the viewer.

7. **Visualise results**

   Render the fused map with static structures in greyscale or a height-based colour ramp, and highlight the robot trajectory and detected objects in contrasting colours.

The project specifications are as follows:

1. **Input rate**

The LiDAR records point clouds at a rate of 10 Hz, but real-time processing is not a requirement for this project.

2. **Languages and libraries**

   Implement the code in Python, using standard point cloud processing libraries such as Open3D, NumPy, pandas, etc.

3. **Alignment accuracy**

   Spatial alignment should be accurate enough to avoid visible misalignment between fused point clouds, with a fitness score of at least 0.60 and an RMSE of at most 0.15 m. These thresholds were empirically defined and are consistent with results obtained under ideal conditions [5].

4. **Map compactness**

   During the alignment and fusion, discard points that already have a neighbour within approximately 5 cm to keep the map light and non-redundant.

5. **Output format**

   Export the final maps as 3D objects (.ply) for better visualization in the software CloudCompare [6].

1.3. Methods and Procedures

This thesis is part of a wider project being carried out at the DEIB department at Polimi. The overall aim is to build a cooperative sensing system that improves situational awareness in vehicular scenarios.

The work revolves around four main goals:

1. **Communication protocol** – design a protocol that exchanges data and synchronises multiple LiDAR-equipped robots so they can share information in real time.

2. **Spatial and temporal alignment** – precisely register point clouds coming from different moving vehicles and keep them synchronised in time.

3. **Dynamic object tracking** – detect, track and register moving objects to help fuse the individual maps.

4. **3D map construction and visualization** – fuse the maps of the different LiDARs and build a unified map that updates continuously

Data is gathered with two LiDAR-mounted robots that belong to DEIB and act as moving vehicles, each equipped with a HESAI PandarXT-32 sensor [7].

Because the communication protocols are still under development, the two LiDAR sensors are currently synchronised manually.

For object detection, the thesis uses a YOLOv8 model designed by a colleague and trained on Roboflow on bird's-eye view images generated from these same LiDAR captures. The Roboflow API is called directly to obtain person and car detections [8].

Point cloud alignment and fusion rely on standard Python libraries such as Open3D, NumPy, pandas and OpenCV. The code implements already defined algorithms such as Iterative Closest Point (ICP) for registration and k-d tree [9] search to evaluate the similarity between point clouds.

Moreover, this project uses PandarView 2 to preprocess the data from the LiDARs and CloudCompare for visualization. The strategy for the alignment of the individual maps takes great inspiration from the KISS-ICP approach [10], adapting its main ideas and functionalities, which are described in more detail in Chapter 2.

1.4. Work Plan

This section will detail the tasks carried out during the project and the milestones achieved, including the time required for each one, to show how the thesis was organised. A Gantt diagram summarising the schedule appears at the end of the section.

The Work Packages are as follows:

WP1. Research – initial study to get familiar with LiDAR SLAM, multi-sensor fusion and YOLO-based object detection.

WP2. **Project Proposal and Work Plan** – write the formal proposal and the detailed work plan.

WP3. **Data acquisition and preprocessing (single LiDAR)** – set up and configure the HESAI PandarXT-32 sensor, collect data and preprocess it using PandarView 2 to obtain usable point cloud frames (.csv).

WP4. Spatial alignment and localisation – develop an algorithm to align sequential point clouds from a single LiDAR to create a consistent and accurate 3D map with the robot's path.

WP5. YOLO object detection integration – integrate YOLO for object detection from the 2D LiDAR images, and convert the detected objects to 3D by associating them with the corresponding points.

WP6. **Critical Design Report** – deliver the mid-project report.

WP7. **Dynamic object removal and map finalization** – remove the dynamic objects detected with YOLO from the fused 3D map, making sure only static elements are retained.

WP8. **Dual-LiDAR map fusion** – develop an algorithm to align and merge point clouds from two LiDARs using detected 3D objects as reference points, creating a consistent fused map.

WP9. **Performance evaluation and optimization** – check system performance, optimise processing speed and accuracy, and validate results.

According to the previous tasks, the milestones are as follows:

WP #	Short title	Milestone/deliverable	Date (week)
2	Project Proposal and Work Plan	Document deliverable	3
3	Data acquisition (single LiDAR)	Collected and preprocessed LiDAR point clouds	5
4	Spatial alignment	Point cloud alignment and 3D mapping	7
5	YOLO integration	2D to 3D object mapping implemented	8
6	Critical Design Report	Document deliverable	8
7	Dynamic object removal	Algorithm removes moving objects from the map	9
8	Dual LiDAR map fusion	Fusion of the two 3D maps	13
9	System validation	Final performance evaluation completed	15

Table 1.1: Work Packages (WPs), milestones, and delivery dates.

Figure 1.1 shows the Gantt chart that visualises these work packages and milestones over the eighteen week timeline.

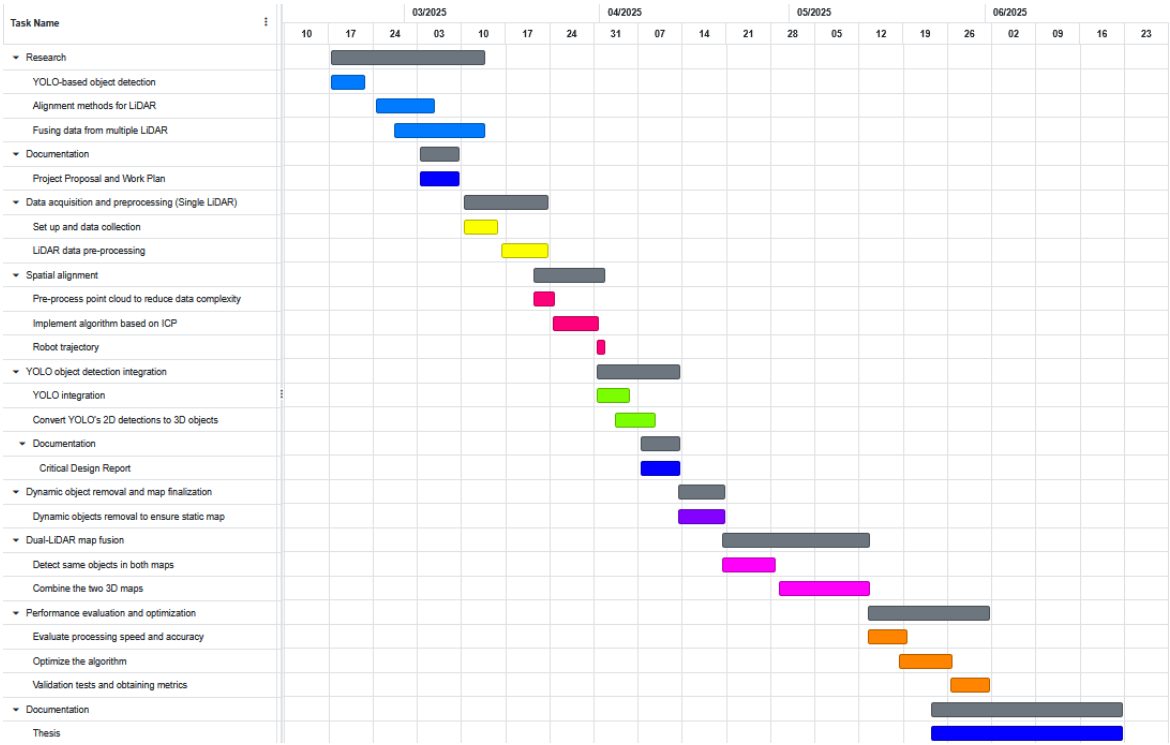


Figure 1.1: Gantt diagram of the project.

1.5. Deviations

Some incidents during development forced a deviation from the original work plan. First, there was a delay in the YOLO training set and the data for the *car* class were not labelled in time, thus the code only uses the *person* class. As a result, the filter of dynamic objects detects and removes pedestrians but may leave moving cars in the static map.

The biggest incident was data collection with both robots at the same time, as it required a supervisor plus two licensed staff members to operate them, and we could not find a time that worked for everyone. Therefore, we had to drop the strategy where a robot follows another robot and fuse the individual maps instead. To test the fusion code, the second dataset is a transformed copy of the first one, simulating different captures. The detailed explanation will be in Chapter 3.

The original plan was to make the YOLO model detect the robots, this way the one following would see the one on the front and vice versa. Then, knowing the relative position between the robots would make the fusion job easier. Due to the previous two incidences the idea lost its purpose, as a robot cannot see itself. Instead, the code now relies on the pedestrian detections to fuse the maps, which could impact the robustness of the algorithm in populated scenes.

CHAPTER 2

State of the Art

Autonomous navigation has progressed rapidly over the past decade. This chapter reviews the three pillars behind that progress: LiDAR-based SLAM, real-time multi-sensor fusion, and object perception. The section outlines how the sensors operate, highlights the leading software and algorithms, and notes the solutions that have been adapted in this project.

2.1. LiDAR Sensing and Hardware

LiDAR (Light Detection and Ranging) determines distance by targeting an object or a surface with a laser and measuring the time for the reflected light to return to the receiver, also known as Time of Flight (ToF) [11].

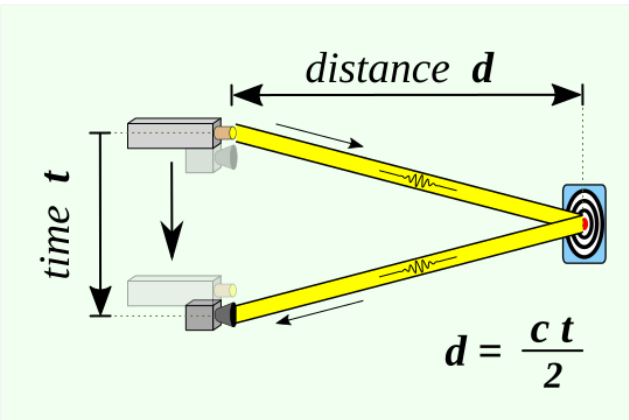


Figure 2.1: Time of Flight principle used in LiDAR systems.

The HESAI PandarXT-32 [7] repeats this cycle thousands of times per second while its head spins, so one revolution produces a full 3D "spray" of points in polar form (r,θ,φ). Converting those points to XYZ results in a point cloud, as shown in Figure 2.2.

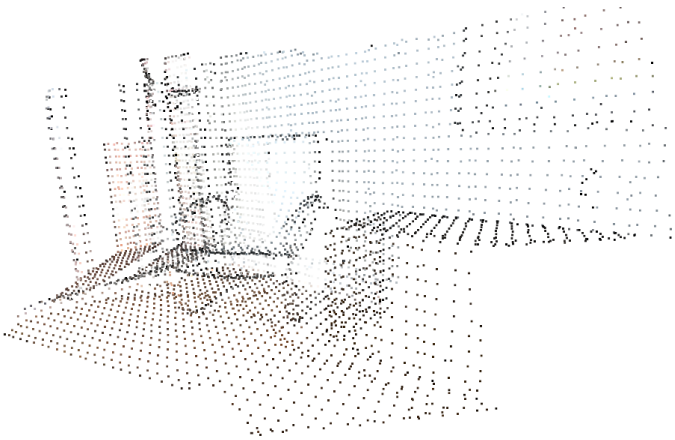


Figure 2.2: 3D point cloud representation of an indoor environment.

The key specifications of the sensor are as follows:

SENSOR	
Scanning method	Mechanical Rotation
Channel	32
Instrument Range	0.05 to 120 m
Range Capability	80 m @10% reflectivity (Channels 9~24)
	50 m @10% (Channels 1~8, 25~32)
Range Accuracy	±1 cm (typical)
Range Precision	0.5 cm (typical)
FOV (Horizontal)	360º
Resolution (Horizontal)	0.09° (5 Hz)
	0.18° (10 Hz)
	0.36° (20 Hz)
FOV (Vertical)	31° (-16° to +15°)
Resolution (Vertical)	1º
Frame Rate	5 Hz, 10 Hz, 20 Hz
Returns	Single Return (Strongest, Last)
	Dual Return (Strongest and Last)

Table 2.1: Key specifications of the HESAI PandarXT-32 LiDAR sensor [7].

The sensor fires 32 laser beams arranged 1° apart in the vertical plane (Figure 2.3). As the head rotates, it takes a sample every 0.09° in azimuth (Table 2.1), so a full sweep collects 128,000 points per cloud, at a frequency of 10 sweeps per second (10 Hz).

(a) Vertical angular distribution of the 32 channels. *(b) Partial Cross-Sectional Diagram.*

Figure 2.3: LiDAR sensor overview.

Each robot is a wheeled platform with the LiDAR mounted on top, alongside some other sensors such as an inertial measurement unit (IMU) [12] (details in Section 3.1).

2.2. Point Cloud Preprocessing

Before registration, detection, or fusion can begin, raw LiDAR data must be converted into a form that is compact, robust against noise, and easy to process. Point cloud preprocessing usually involves three main steps:

1. **Frame decoding** – PandarXT-32 packets contain range, azimuth, vertical angle and reflectivity values and are encoded within a PCAP file (the standard container for raw network traffic). There is no convenient real-time approach for decoding these packets and converting them to usable point clouds in Python, so the first step is to transform them into XYZ point clouds. Hesai's official tool, PandarView 2 [4], converts each sweep into a comma-separated CSV file, which are much easier to load and process.

```
Ch,AziCorr(deg),Dist(m),Rfl,Azi(deg),Ele(deg),t(us),x(m),y(m),z(m),
1,224.820007,9.2360,12,224.820007,15.000000,662248,-6.288455,-6.328092,2.390453,
2,224.820007,9.1760,9,224.820007,14.000000,662248,-6.275868,-6.315425,2.219875,
3,224.820007,9.1360,10,224.820007,13.000000,662248,-6.274749,-6.314299,2.055153,
4,224.820007,9.1000,11,224.820007,12.000000,662248,-6.274255,-6.313801,1.891996,
5,224.820007,9.0680,13,224.820007,11.000000,662248,-6.274432,-6.313979,1.730256,
```

Figure 2.4: Example of a CSV file generated by the LiDAR sensor.

2. **Outlier rejection** – Reflections from raindrops, exhaust fumes and multipath echoes often leave single, isolated points. A statistical outlier removal (SOR) filter [13] drops any point whose mean neighbour distance exceeds $\mu + k\sigma$. Typical values of 20 to 50 neighbours and 1.0 to 2.0 standard deviation strike a good balance between removing sparse points and preserving thin structures such as poles or tree trunks [14].

3. **Geometric downsampling** – Running ICP or YOLO on millions of points per frame is computationally demanding and often unnecessary, because many points are either repeated or extremely close to one another. A voxel grid filter (where a voxel is the 3D equivalent of a pixel) [15] keeps just one representative point per cell. Because no averaging is being performed, this preserves sharp edges and can reduce around 90% of the data, which speeds up alignment without sacrificing map fidelity.

2.3. Registration Techniques

To merge successive frames or even entire maps, the point clouds must be expressed in a common coordinate system. Registration is the process of finding the rigid transformation $T = [R|t]$ (rotation R + translation t) that superposes one cloud onto the other while minimizing the distance between corresponding points [1].

Over the last two decades the community has introduced many registration methods, each balancing accuracy, robustness and speed. This section summarises the most widely used approaches.

2.3.1. Nearest-Neighbour Search: K-D Tree

A k-d tree is a binary space-partitioning data structure that speeds up nearest-neighbour queries in point clouds [9].

To build the tree, we take all the points, choose the axis with the widest spread (in 3D that is x, y or z), split the cloud at the median value of that axis, and recurse on the two halves. Each split creates an internal node, and the process stops when a leaf contains only a handful of points. The resulting tree has a depth of roughly $\log_2 N$ [9].

To find the closest point to a query point p, the search descends the tree along the branch that contains this point, examines the leaf, and then backtracks, discarding any branches whose bounding box is farther than the best distance found so far. On average this costs $O(\log N)$ instead of the $O(N)$ required by a brute-force scan [9].

Additionally, the algorithm provides two metrics [5]:

1. **Fitness score** – the fraction of source points whose closest match lies within a user-defined threshold. A higher score indicates greater overlap between the clouds.
2. **RMSE (Root-Mean-Square Error)** – the average alignment error computed over the inlier pairs. A lower RMSE indicates a more precise registration.

This algorithm is crucial for this thesis because ICP calls it thousands of times per frame, then a new k-d tree is built when each new cloud is added to the existing

static map to filter already existing points, and it is used once more during map fusion to find the best correspondences between the two robots' clouds.

2.3.2. Point Cloud Alignment Methods

Once the nearest neighbours are known, the next step is to find the rigid transformation that best overlays two point clouds. A classic tool for this is Iterative Closest Point (ICP) [1]:

1. **Match** – for every point in the source cloud, pick its closest neighbour in the target cloud (which is fast due to the k-d tree) [9].
2. **Transform** – solve for the rotation R and translation t that minimise the Euclidean distance between each matched pair:

$$E(R,t) = \sum_i \|Rp_i + t - q_i\|^2 \text{ [16]}$$

3. **Iterate** – apply $T = [R|t]$ to the source, rebuild the matches, and repeat until the RMSE falls below the chosen threshold or until a maximum number of iterations is reached [5].

There are two common ICP variants [17]:

1. **Point-to-point** – minimises the raw Euclidean distance.
2. **Point-to-plane** – projects the error onto the normal of the target point, often converging in fewer iterations when reliable normals are available.

The biggest drawback of ICP is that a bad initialisation can prevent convergence or trap the optimiser in a local minimum. To reduce that risk, some modern variants have been proposed:

1. **GICP (Generalised ICP)** – blends point-to-point and point-to-plane costs. Better performance with mixed geometry, but slower [18].
2. **NDT (Normal Distributions Transform)** – replaces target points with Gaussian cells on a voxel grid. Better for clouds with different densities or little overlap, but requires more computation [19].

All the variants rely on fitness score and RMSE to decide whether to accept the results or to keep iterating. The point clouds captured with the PandarXT-32 sensor are moderately dense but become noisy at long range, so computing reliable normals would cost extra time and memory. A plain point-to-point ICP offers the simplest code path, runs comfortably on a laptop CPU, and already meets the requirements defined in the previous chapter. Chapter 3 explains how the initialisation has been improved to address its main limitation.

2.3.3. Feature Extraction: FPFH + RANSAC

A popular alternative to ICP (or complement) is to align clouds using geometric features rather than the raw coordinates of the points. The standard pipeline combines these two components:

1. **FPFH (Fast Point Feature Histogram)** – for each key point, the algorithm looks at its neighbours, computes their surface normals, and stores the resulting inclination angles in a histogram [20]. Because the histogram captures relative geometry, it remains the same no matter how the cloud is rotated, so two similar histograms are likely the same kind of corner or edge.
2. **RANSAC pose estimation** – after matching many of these FPFH histograms between the two clouds, RANSAC picks three matched points at random, computes the rigid move (rotation + translation) that aligns them, and checks how many other matches agree with that transformation [21]. After doing this a few hundred times, the transformation that earns the most votes is kept, and then ICP is then applied to tune the result.

This feature-based approach is useful when overlap is small or the initial pose is unknown, because it can "find" the distinctive shapes, such as corners and edges. In principle, it could serve as a fusion step for our two maps, but we left it aside for two reasons. First, the project already relies on the YOLO detector to match objects between the robots, so adding another detector would complicate the pipeline without a clear gain. Second, the FPFH + RANSAC block behaves like a "black box", producing results that are hard to debug when something goes wrong.

2.4. LiDAR SLAM for Trajectory Estimation

Simultaneous Localisation and Mapping (SLAM) is the principal component that lets a robot know where it is while it is still building the map. With LiDAR, each new scan is aligned to what the robot already knows, and the resulting pose is accumulated to the previous ones to make a trajectory. This project builds on the core idea of the KISS-ICP paper, which breaks the process into four steps [10]:

1. **Motion compensation** – a single frame takes 100 ms to be scanned, meaning the robot has already moved in this time, causing motion distortion. KISS-ICP assumes constant velocity from the previous poses to estimate that motion and correct the frame.
2. **Scan to map alignment** – the current cloud is downsampled and matched to the continuously growing map using ICP.
3. **Local map update** – aligned points go into the k-d tree map, but any point that has a neighbour within a certain radius (typically 5 cm) is skipped, keeping the map smaller and less redundant.

4. **Pose chain** – each new transformation is chained to the past ones, producing a running trajectory of the robot's path.

Other, more advanced SLAM approaches mix LiDAR data with other sensors to improve the pose estimate, known as odometry. A more accurate trajectory allows for better motion compensation and initialisation, and therefore, improves alignment.

The two most common sensors are:

1. **IMU (Inertial Measurement Unit)** – measures linear acceleration and angular rate with accelerometers and gyroscopes (and sometimes a magnetometer) [22][12].
2. **Wheel odometry** – estimates the position and orientation of a wheeled robot by tracking the rotation of its wheels [23].

The robots in this thesis are equipped with IMUs, but these sensors were not available when the first data were collected. The plan was to obtain IMU data during the dual-robot recording session, but since that couldn't take place, this idea had to be dropped.

2.5. Object Detection on LiDAR Data

LiDAR point clouds can be analysed for objects in several ways. The three most common are:

1. **Voxel-grid networks** – the cloud is quantised into a 3D grid and processed with sparse 3D convolutions (e.g. SECOND, HVNet) [24][25]. The full geometry is preserved and the network reasons over real volumes, but memory grows cubically with resolution and requires a lot of computing power [24].
2. **Bird's-eye view CNNs** – the cloud is flattened onto the ground plane, and simple per-cell statistics such as hit count and maximum height are stored (e.g. PointPillars, YOLO-BEV) [26][27]. The result is a compact image that any fast 2D detector can handle, at the cost of losing the vertical dimension.
3. **Range-image CNNs** – the 3D scan is unwrapped into a 2D panorama in which each row corresponds to a channel and each column the azimuth (e.g. RangeDet) [28]. Like the BEV approach, the output is a regular image that any 2D detector can process.

Each family has its own trade-offs (speed, memory, accuracy), but selecting among them is outside the scope of this project. Because a teammate already trained a YOLOv8 model on BEV images generated from the PandarXT-32, this thesis adopts the detector as a fixed starting point.

2.5.1. YOLO-Based Object Detection

YOLO (You Only Look Once) [3] is a popular object-detection model for images and videos, known for its speed and solid accuracy.

Object detectors fall into two groups: two-stage and one-stage models [29]. YOLO belongs to the one-stage type, meaning it processes an image in a single pass. This makes it very fast, though usually slightly less accurate than two-stage methods [30].

YOLO works with an end-to-end neural network that makes predictions of bounding boxes and class probabilities all at once, with the help of a single fully connected layer. First, it slices the input image into an S×S grid. For every cell the network predicts a small vector that contains the centre offset and size of a few candidate boxes, and a confidence score that says how likely it is that an object of a given class is present. Then, boxes with low confidence are dropped, and the rest go through non-maximum suppression, so overlapping boxes of the same class become a single detection [3].

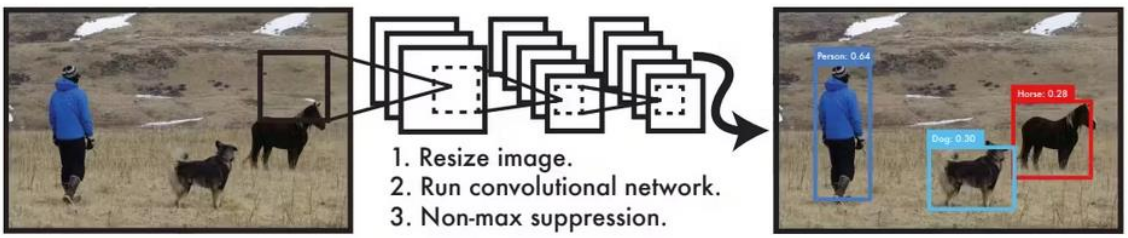


Figure 2.5: YOLO's object detection pipeline.

In this project, each LiDAR frame is first converted to a bird's-eye view image for detection. The resulting bounding boxes are then converted back to the 3D space to find the matching points in the cloud. The full description of the process is given in Chapter 3.

2.6. Multi-LiDAR Map Fusion

Early research focused on a single scanner mounted on one vehicle. Because sending thousands or even millions of points per second is demanding, fusing the views of multiple LiDARs in real time is still a recent and active field. Current approaches can be grouped into three categories:

1. **Target-based calibration** – a known object that every scanner can see is placed in the scene (e.g., a checkerboard). Once the target is detected in each cloud, aligning the sensors becomes straightforward. The drawbacks are that it requires manual setup and recalibration when the target is moved [31].
2. **Geometry-only alignment** – each map is treated as an ordinary point cloud, and the two clouds are fused with a registration algorithm such

as ICP, GICP or NDT. This method struggles when the maps have little overlap, and it also requires a good initialisation [32].

3. **Semantic-guided fusion** – uses detected objects as anchors, visible by both LiDARs. It is very robust against sparse overlap and does not need calibration, but it requires a reliable detector [33].

Because real-time performance is not required and bandwidth is not a factor in this thesis, that constraint can be ignored. Since a YOLO model [3] that detects people in every scan is already available, the most straightforward approach is to fuse the maps by using those detections as anchors.

CHAPTER 3

Methodology and Project Development

This chapter describes the approach followed to develop the project, covering both software and hardware aspects. It outlines the main steps taken to process and align LiDAR data, detect moving objects, estimate robot trajectories, build the maps and finally fuse the results from the two sensors into a unified map of the environment.

The first section covers the experimental setup and how the LiDAR data is converted into point clouds. Then it explains the full pipeline for a single robot, including alignment and object detection. After that it describes the strategy used to fuse the maps from two sensors, and the final section discusses the main challenges and limitations encountered during the project.

3.1. Experimental Setup

Each test run uses a four-wheeled robot owned by the DEIB department at Polimi (Figure 3.1(a)). A Hesai PandarXT-32 LiDAR is mounted on an aluminium structure about 1 m above the ground, giving the sensor a clear view of pedestrians and even car roofs.

During recordings, the scanner is connected to a laptop for data capture and powered from an onboard battery. The platform also carries an IMU, but its data could not be exported and are therefore not used in this thesis.

As explained in earlier chapters, the plan was to have one robot follow another a few metres behind. Because that session could not take place, all measurements were taken with a single robot. Four captures were recorded: one inside a DEIB hallway and three on the street just outside the building.

(a) Wheeled robotic platform equipped with a LiDAR sensor.

(b) Outdoor data acquisition near the DEIB building using a single LiDAR-equipped robot.

Figure 3.1: Experimental setup for data acquisition.

3.2. Pipeline Overview

The project is structured in two main stages: a single-robot SLAM pipeline and a dual-LiDAR fusion that builds upon it. Figure 3.2 shows the processing flow for a single LiDAR, from raw data to the final map and trajectory. Figure 3.3 extends this setup to explain how the two captures are fused into a shared map.

Single-robot SLAM pipeline:

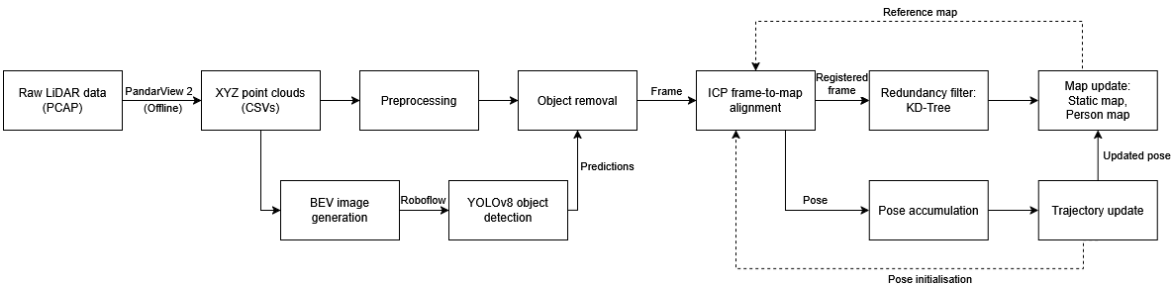


Figure 3.2: Single-robot SLAM block diagram.

The process begins offline by converting a raw PCAP file into point clouds with PandarView 2. Each frame is then downsampled and filtered to reduce noise and improve processing speed. A BEV image is generated and passed through the YOLOv8 model trained to detect pedestrians. The detected objects are removed to build a static map.

The remaining points are aligned to the reference map using point-to-point ICP, with the previous pose as an initial estimate. The resulting transformation is accumulated over time, the new points are added to the map and the trajectory is updated.

Dual-robot fusion pipeline:

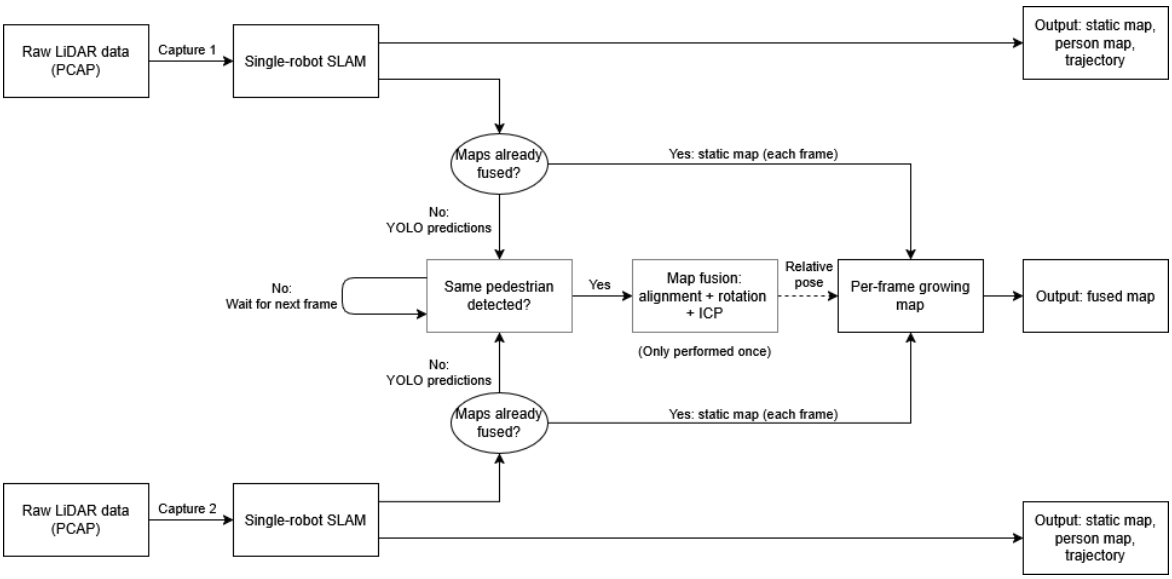


Figure 3.3: Dual-robot map fusion block diagram.

The same SLAM pipeline runs independently on both captures. When both sensors detect the same pedestrian, this triggers the fusion process. The second map first is translated so the detected objects overlap, then the best rotation is estimated, and finally ICP refines the alignment. This process is performed once.

From that point onwards, each new frame is aligned using the previously computed transformation, and both maps are continuously merged into a unified view of the environment.

Each stage shown in the diagrams is explained in detail in the following sections.

3.3. Preprocessing, Single-Robot SLAM and Semantic Detection

Before implementing multi-robot fusion, the first step is to build a complete SLAM pipeline for a single robot. The goal is to convert the LiDAR recording into a clean 3D map, removing dynamic objects and reconstructing the robot's trajectory.

To achieve this, a script was developed that handles one LiDAR capture: processes each frame, detects pedestrians using a YOLOv8 model, aligns the point clouds using ICP, and builds the map. The implementation was done in Python as a requirement, and it uses common libraries such as **Open3D** for 3D point cloud processing, **NumPy** for matrix operations, **pandas** for handling data, and **OpenCV** (cv2) for image manipulation.

This script serves as the foundation for the dual-robot version developed in Section 3.4, where most functions are reused. The following sections explain the full pipeline, from decoding the raw data to building the final maps.

3.3.1. Packet Decoding

The PandarXT-32 stores each recording as a PCAP file containing raw network packets. Since no convenient real-time decoder is available for Python, the first step is to convert the file into usable point clouds with Hesai's PandarView 2 software.

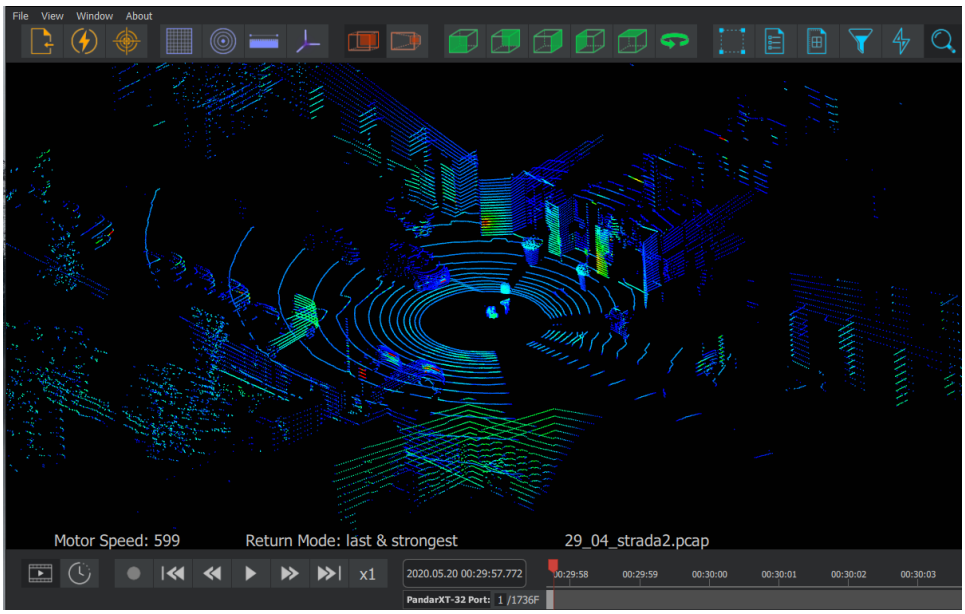


Figure 3.4: PandarView 2 software showing a frame from a PandarXT-32 PCAP capture in an urban environment.

Figure 3.4 shows PandarView 2 with a PCAP capture loaded, displaying the first frame. The software's export function saves each frame as a CSV file containing the XYZ coordinates of each point, along with additional data such as intensity. In the example shown, the PCAP file contains 1,737 frames, corresponding to around 3 minutes of recording. After conversion, the data expands from 938 MB to 16.7 GB of CSVs. Although the conversion process takes about one hour on a standard laptop, it only needs to be performed once and facilitates reusing and sharing the data.

3.3.2. Noise Filtering and Downsampling

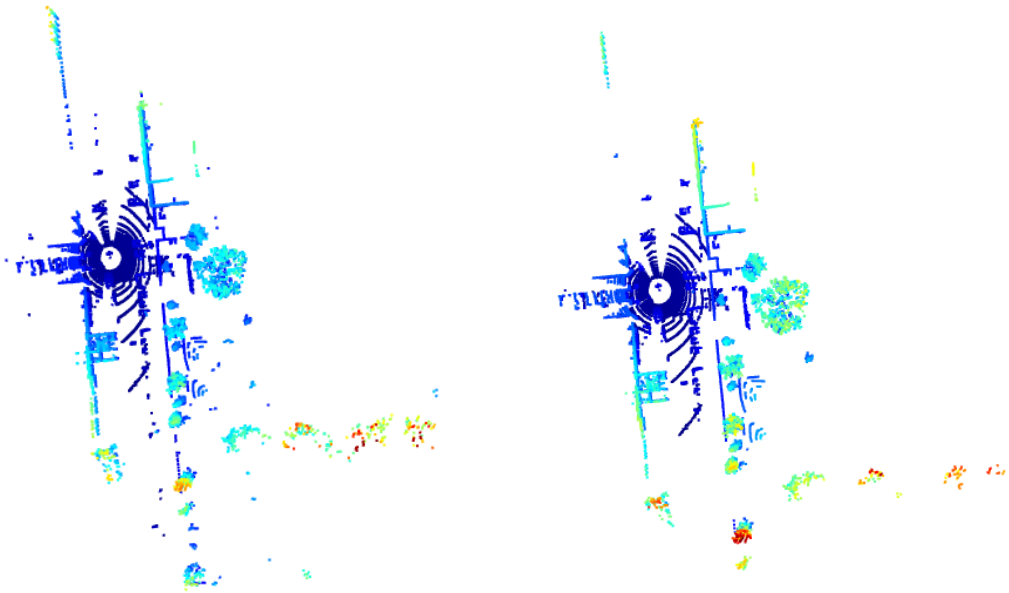
Raw point clouds captured by the LiDAR often contain noise, outliers, and an excessive number of points, especially in outdoor environments where reflective surfaces or distant objects can introduce measurement errors. To improve the quality and efficiency of the next processing steps, two preprocessing techniques are applied:

1. **Downsampling** – to reduce computational load while preserving the overall structure of the point cloud, voxel grid filtering is applied. This method divides the space into 3D grid cells (voxels) and takes a single representative point per voxel, typically the centroid. A voxel size of 0.1

meters was selected, which is commonly recommended for large-scale environments [34].

2. **Noise filtering** – due to reflections or poor detections, point clouds often contain isolated points. A Statistical Outlier Removal (SOR) filter is used to eliminate these by analysing the neighbours of each point and removing the ones that deviate from the mean distance. A configuration of 20 neighbours and a standard deviation of 2.0 was selected, as it provides a good balance between removing noise and preserving thin structures such as poles or tree trunks [14].

These two steps are essential before applying alignment methods such as ICP. Figure 3.5 shows the original point cloud and the result after applying downsampling and noise filtering. The original cloud contained 128,000 points, which was reduced to 28,810 after applying a voxel downsampling with a 0.1 m voxel size, and 28,638 after noise filtering. As shown, the overall shape of the cloud is preserved, while isolated points are removed, noticeable in the tree region at the bottom right.



(a) Raw point cloud. *(b) Point cloud after downsampling and noise filtering.*

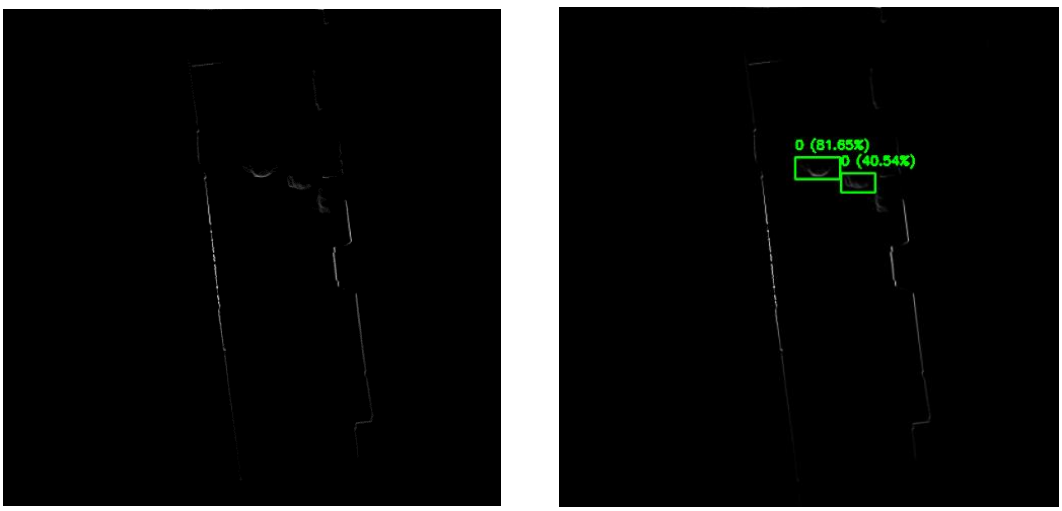
Figure 3.5: Point cloud before and after preprocessing.

3.3.3. YOLO-Based Object Detection

Since a colleague has trained a YOLOv8 model, we adopt it for the detection of the objects inside the point clouds. Each LiDAR frame is turned into a bird's-eye view (BEV) image that the YOLO detector can read. The process has four steps:

1. **BEV image generation** – a 10×10 m region around the sensor (the choice of size is explained in Section 3.5) is binned into a 600×600 grid, which gives a resolution of 1.67 cm/px. We store the hit count per cell, where ground cells show few hits and vertical objects (e.g., a person) have many. The result is a greyscale image: cells with high counts are white, while low counts are darker, making objects stand out.
2. **Inference call** – the YOLOv8 model was trained on Roboflow on the same LiDAR's captures. Each image is sent to Roboflow's API, which returns the centre, width and height of the predicted boxes. Boxes with confidence below 0.10 are discarded (the reason for that threshold is in Section 3.5).
3. **Back-projection** – for every box, we map the coordinates back to the 3D space to get all the points inside. Since the captures were recorded outdoors, there is nothing above the pedestrians, but we could limit the box height to about two metres.
4. **Map update** – since the model currently detects only pedestrians, we treat every box as a moving object. Objects are removed to keep a static map (grey) and added to a separate cloud with people (green), for better visualization.

Figure 3.6 shows the image generated and the predicted boxes:

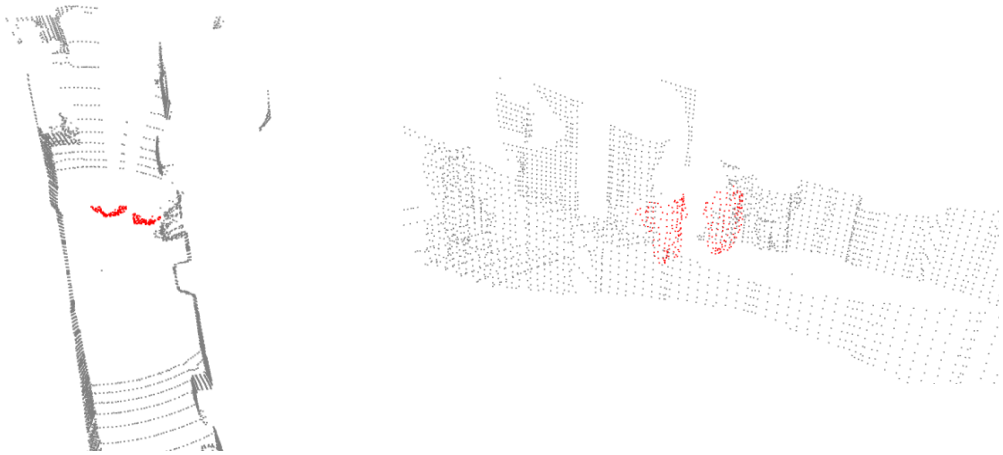


(a) BEV image of an indoor hallway showing three people.

(b) Same image with predicted bounding boxes, only two people detected.

Figure 3.6: Bird's-eye view images from an indoor scan.

Figure 3.7 shows the matching 3D points:



(a) Top view of the point cloud showing points matched to detected objects (in red).

(b) Side view of the point cloud showing points matched to detected objects (in red).

Figure 3.7: Matched 3D points for detected objects.

3.3.4. Scan-to-Map ICP and Pose Chain

After removing dynamic points, the filtered cloud is aligned to the current static map using point-to-point ICP. As explained in Chapter 2, other methods require more computation, while ICP offers the simplest code path and already meets our requirements (Section 1.2).

1. **Initial guess** – since robot motion is nearly constant between frames, we start ICP with the best transformation found in the previous frame. This gives the algorithm a good initialisation.
2. **Settings** – 0.3 m inlier radius and 50 maximum iterations. Since the LiDAR scans at 10 Hz and the robot moves below 3 m/s, this threshold is large enough to include valid matches without checking further points. ICP stops early if the change in fitness and RMSE falls below 1×10^{-4}, meaning the solution is no longer improving.
3. **Output** – ICP returns the rigid transformation, the RMSE, and the fitness score, which will be analysed in Chapter 4.

The rigid transformation T_k found for frame k is chained to the running pose estimate:

$$T_{0\rightarrow k} = T_{0\rightarrow k-1}T_k$$

Where T_k is the transformation returned by ICP for frame k, $T_{0\rightarrow k-1}$ the global transformation accumulated up until the previous frame, and $T_{0\rightarrow k}$ the new pose.

Linking all these transformations builds a continuous trajectory, which provides the robot's localisation over time.

3.3.5. Map Update and Map Export

Once the current cloud is aligned, it is merged into the static map. Before adding new points, a k-d tree checks whether there is a neighbour closer than 5 cm. If so, the point is skipped. This keeps the map light and avoids extra computation in later frames.

At the end of the run (or after the chosen number of frames), the program saves three PLY files:

- Static map: contains only fixed structures.
- Objects: all detected pedestrians.
- Trajectory: the robot's path.

These files can be opened directly in CloudCompare for easy visualisation.

3.4. Dual LiDAR Fusion

After validating the single-robot SLAM pipeline, a second script was developed to handle two LiDAR datasets simultaneously, with the goal of fusing both maps into a unified view of the scene.

Due to the limitations explained earlier, the dual-robot recording session could not take place. As a workaround, we duplicated one dataset and applied a 180° rotation offline to simulate a different capture (the reason for this angle is explained in Section 3.5).

Although a translation might seem necessary, it is not. Each CSV file stores point coordinates relative to the LiDAR sensor itself, meaning that the origin of the cloud is always at the scanner's position. Translating the second dataset would change this origin and break the internal consistency of the data, especially for the bounding boxes predicted by YOLO, which expect the sensor to be at the centre. Correcting this would require adjusting every prediction individually, adding complexity without providing any benefit in this simulation.

The script begins by applying LiDAR SLAM independently on each dataset. Once a shared detection (in this case, a pedestrian) is identified in both scans, it serves as an anchor for estimating the relative pose between the two coordinate systems. The fusion process is divided into three stages.

3.4.1. Map Alignment Using Bounding Boxes

The first step assumes that the pedestrian detected in both frames corresponds to the same person (the justification for this assumption is explained in Section 3.5). Since each point cloud is centred around its own sensor, the coordinates of the bounding boxes are also relative to that origin. By taking the centre of each bounding box and computing the vector between them, we can translate the second cloud so that the pedestrian in both maps align in the same position.

The scenario is illustrated below:

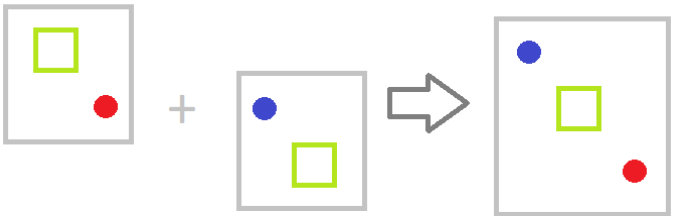


Figure 3.8: Fusion based on pedestrian detection. The second map is translated so that the detected pedestrian (green) aligns in both point clouds, with LiDAR 1 shown in red and LiDAR 2 in blue.

On the left, the two individual maps before fusion. On the right, the second map has been translated so the bounding boxes' centres overlap. Green boxes show the detected pedestrian, the red circle is the first sensor, and the blue circle the second sensor.

After this translation, both maps are centred around the same object. The only transformation missing is the rotation, which in this case is the 180° applied initially.

3.4.2. Rotation Search Using K-D Tree Fitness Score

Once both maps are in the same coordinate system, the next step is to find the best rotation for the second map. To do this, we rotate it step by step from 0° to 360°, in 1° increments (this parameter can be adjusted, larger steps are faster but less accurate).

For each rotation, we use a k-d tree to compute the fitness score, which measures the proportion of points in one cloud that find a close match in the other cloud. The rotation angle with the highest score is selected.

This step provides an initial but effective rotational alignment between the two clouds, which is refined in the third step.

3.4.3. ICP Refinement

Since the bounding boxes might not be exact and the selected rotation might also be slightly off, we apply a final refinement using point-to-point ICP between the two clouds. This step finds the optimal rigid transformation, improving the alignment before merging both maps.

After computing the translation from the bounding boxes, the best rotation, and applying the ICP refinement, we now know the relative pose between the two sensors. From this point, we chain the three transformations and apply them to the static map from the second LiDAR to align it with the first one. This step only needs to be done once.

For all the following frames, we apply the same transformation to the second cloud and merge both maps. In this test case, the result should ideally match the first map exactly, since both datasets come from the same recording. However, in a real scenario, the fused map would combine two different perspectives into a single, more complete view of the environment.

3.5. Limitations and Mitigation

During the development of the project, some limitations affected the results or prevented the implementation of certain ideas. This section outlines the main issues and how they were handled or could be improved in future work.

3.5.1. No Dual-Robot Capture

The original idea was to record both robots moving together, with one following the other. However, due to scheduling and logistics constraints (requiring a supervisor and two licensed operators), this session could not be arranged.

As a workaround, the dataset was duplicated and a 180° rotation was applied offline to simulate a second capture. This allowed testing the fusion process using a single recording. The code would still work with real dual-robot data, with minor adjustments.

3.5.2. YOLO Model

The biggest limitation has been the YOLO object detector. First, it only detects pedestrians, not cars. Second, it was trained using the same LiDAR captures, where the operators walk behind the robot.

Initially, the idea was to rotate the point clouds by 30 to 45° to simulate the second capture, which would be closer to the real scenario. However, the model struggled to detect pedestrians when the point of view changed, since it had not been trained with these perspectives. The 180° rotation gave the best results, which is why it was chosen.

Also, the BEV image is generated from a 10 × 10 m area around the sensor because the model was trained with these parameters. Increasing this region significantly reduces accuracy. In some captures, the operators were further than 5 metres away from the robot and were not detected, resulting in extra noise in the final map.

The confidence threshold had to be lowered to 0.10 because many detections fall below 15%. Even with this setting, many pedestrians are still not detected, and sometimes there are false positives such as car bumpers.

3.5.3. Anchor Pedestrian

The original idea was to detect the robots themselves using YOLO, so they could be used as anchors when fusing the maps. Since the robots could not be captured together, the YOLO model could not be trained, and this approach had to be discarded.

In the current implementation, the detected pedestrian is assumed to be the same person in both maps. This assumption is not guaranteed, but in the real setup the robots would only need to detect each other once, avoiding ambiguity.

3.5.4. IMU Data

Although the robots are equipped with IMUs, this data was not available. This limits the ability to perform motion compensation, especially in scenarios involving fast or sharp movements. In such cases, algorithms like ICP may struggle to align the frames correctly, leading to mapping errors. Access to IMU data could have improved initialisation and alignment.

CHAPTER 4

Results

This chapter presents the experimental results obtained during the development of the project. The goal is to evaluate the performance of the SLAM pipeline and the dual-LiDAR fusion strategy.

These results are divided into three main sections:

1. The first section outlines the tests conducted and the conditions under which they were recorded. It also presents key evaluation metrics.
2. The second section provides visual representations of the outputs, including object detection, trajectories, and final maps.
3. The third section discusses the main limitations encountered when interpreting the results.

The results help validate the implementation, showing what works well and what could be improved.

4.1. Experiments and Tests

To evaluate the performance of the proposed SLAM pipeline and the fusion method, four recordings were used, all captured with a Hesai PandarXT-32 LiDAR mounted on a robot. One recording was done indoors in a corridor at the DEIB department (Politecnico di Milano), and the remaining three were outdoors on the same street.

Each capture was processed using the single-robot SLAM pipeline described in Section 3.3, which includes dynamic object detection, map construction with ICP alignment, and trajectory estimation. The dual-robot fusion method was tested by duplicating each dataset and applying a 180° rotation, as explained in Chapter 3.

4.1.1. Test Conditions

Table 4.1 summarises the test conditions for all four recordings, including the environment, duration, number of frames, and raw data size for each run.

Test ID	Environment	Duration (s)	Frames	PCAP size (GB)
1	Indoor hallway	675	5,016	2.65
2	Outdoor street 1	125	1,256	1.39
3	Outdoor street 2	237	1,737	0.938
4	Outdoor street 3	344	3,434	1.81

Table 4.1: Test runs used for SLAM and fusion evaluation.

4.1.2. YOLO Detection Performance

The object detection stage was carried out using a YOLOv8 model trained on BEV images from the same LiDAR captures. Each frame was converted into a 600×600 greyscale image and sent to Roboflow's API, which returned the predicted bounding boxes for pedestrians.

The model performed well in the indoor test (Test 1), where the data had been included in the training set. However, results in the outdoor captures (Tests 2, 3, and 4) were less consistent.

A key factor affecting detection accuracy was the orientation of the pedestrian's body in the BEV images. As shown in Figure 4.1, pedestrians appearing horizontally aligned within the image were generally detected successfully (Figure 4.1(b)). In contrast, when pedestrians appeared diagonally, their shape was less defined and they were often missed by the model (Figure 4.1(a)).

(a) BEV image showing diagonally oriented pedestrians that the model missed.

(b) BEV image showing horizontally oriented pedestrians correctly detected.

Figure 4.1: Bird's-eye views illustrating how pedestrian orientation affects detection performance.

To compensate for low confidence scores, the detection threshold was lowered to 0.10, as many pedestrian predictions fell below 15%. This affects the quality of the

static map and the fusion process, as missed detections leave dynamic objects in the map, while false positives remove valid points.

4.1.3. SLAM Alignment

To evaluate the performance of the SLAM algorithm for a single robot, we recorded the RMSE and fitness scores at each frame during the alignment process. Test 3 was selected as a representative case, since it includes a long outdoor capture with multiple pedestrians, parked cars, and sharp robot turns, making it a good test of robustness.

Table 4.2 shows the average fitness score and RMSE for every 100 frames in Test 3.

Frame interval	Average Fitness Score	Average RMSE (m)
0–99	0.984	0.079
100–199	0.988	0.114
200–299	0.995	0.116
300–399	0.991	0.119
400–499	0.955	0.126
500–599	0.955	0.128

Table 4.2: ICP alignment statistics for Test 3 (averaged every 100 frames).

As shown, the alignment error increases significantly after frame 400, which corresponds to a section where the robot makes multiple fast and sharp turns. In these situations, point-to-point ICP alone cannot fully correct the motion distortion.

One possible way to mitigate this would be to increase the ICP inlier radius, although this would also increase computation time for all frames. An alternative would be to align each frame to a local map built from the last few frames, which could help reduce accumulated error and improve robustness.

In addition to the fitness and RMSE metrics, several parameters were recorded during execution, such as processing time, number of points per frame, and map growth. These are analysed and visualised in Section 4.2.

Results for the other three test cases are included in Appendix A.

4.1.4. Map Fusion

To evaluate the multi-robot fusion strategy, the method was tested on all four datasets by simulating a dual-robot scenario. As explained in Section 3.4, each dataset was duplicated and rotated by 180° rotation to simulate a different capture and test the fusion pipeline.

Fusion is triggered when both sensors detect the same pedestrian. First, the centres of the predicted bounding boxes are aligned. Then, the second map is rotated in 1° increments to maximise overlap, based on a k-d tree fitness score. Finally, point-to-point ICP is applied to refine the alignment.

To assess the accuracy of the bounding box alignment, the translation error is computed as the Euclidean distance between the centre coordinates of the two detected boxes:

$$Translation\ error = \sqrt{(x_1 - x'_2)^2 + (y_1 - y'_2)^2}$$

Where $(x'_2, y'_2) = (-x_2, -y_2)$, due to the initial rotation applied to the second dataset.

Test ID	Fusion start frame	Original box centre (x, y)	Rotated box centre (x, y)	Box translation error (m)	Rotation found (°)	ICP RMSE (m)
Test 1	2	(1.458, 0.325)	(-0.783, -0.875)	0.871	178	0.0609
Test 2	120	(-1.812, 4.95)	(0.633, -4.89)	1.185	173	0.074
Test 3	28	(-0.183, 1.033)	(0.225, -0.942)	0.101	178	0.065
Test 4	0	(2.583, -2.475)	(-2.417, 1.458)	1.030	179	0.066

Table 4.3: Accuracy metrics for the dual-LiDAR fusion simulation.

RMSE is used as the main evaluation metric in this section because it reflects geometric alignment error after refinement, independently of the threshold used for the correspondence matching. This makes it more consistent across all the datasets.

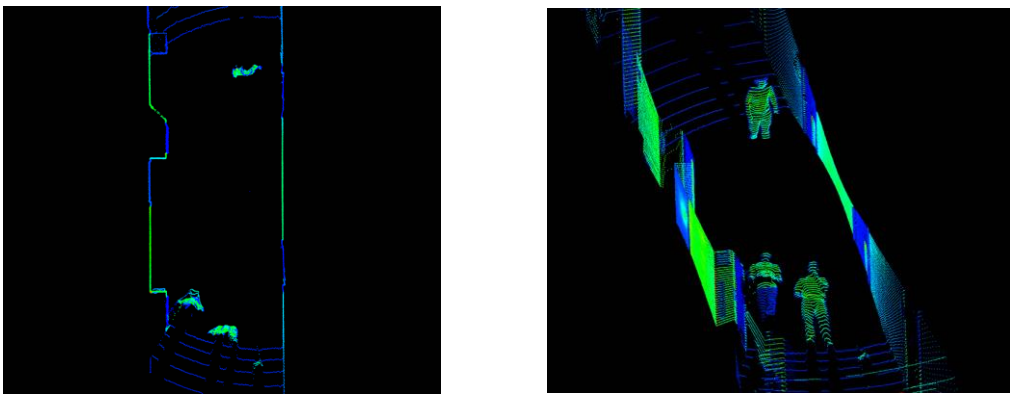
Ideally, if YOLO detected the same pedestrian identically in both maps, the translation error would be 0 and the optimal rotation would be 180°. In practice, small differences in detection accuracy, combined with variations in the removal of dynamic objects, lead to slight deviations between the two static maps. Despite these issues, the fusion process still achieves RMSE values below 0.10 m, meeting the specifications defined in Section 1.2 and validating the proposed alignment method.

4.2. Data Visualization

This section presents visual representations of results obtained during this project. First, we validate YOLO detections in 2D and their projection into 3D. Next, we present the static map, the pedestrian map and the robot's trajectory. Then, we plot key SLAM metrics per frame, such as RMSE, fitness score and point counts. Finally, we display the fused map combining both LiDARs.

4.2.1. Semantic Mapping Validation

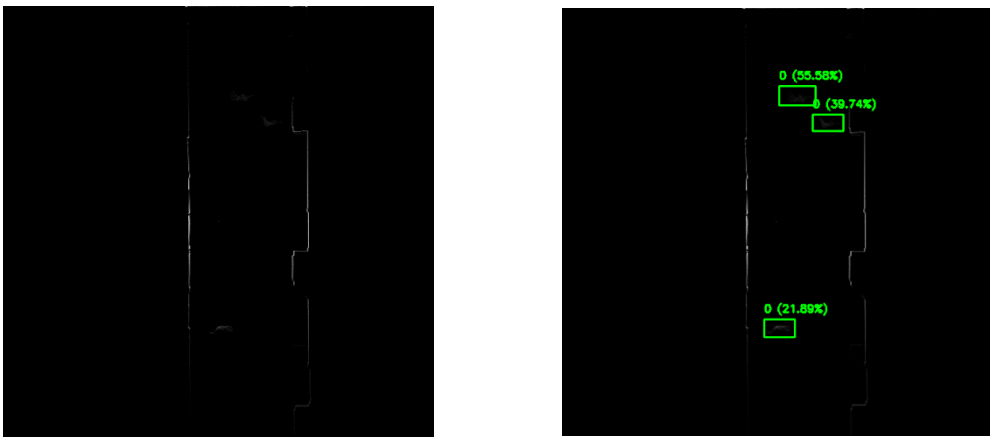
First, we verify that YOLOv8 detections in 2D correctly map back into the 3D point cloud. Figure 4.2 show the raw LiDAR frame in PandarView 2, where three pedestrians are visible in the original scan.



(a) Top view of the raw point cloud. *(b) Side view of the raw point cloud.*

Figure 4.2: Raw LiDAR point cloud views with three pedestrians present.

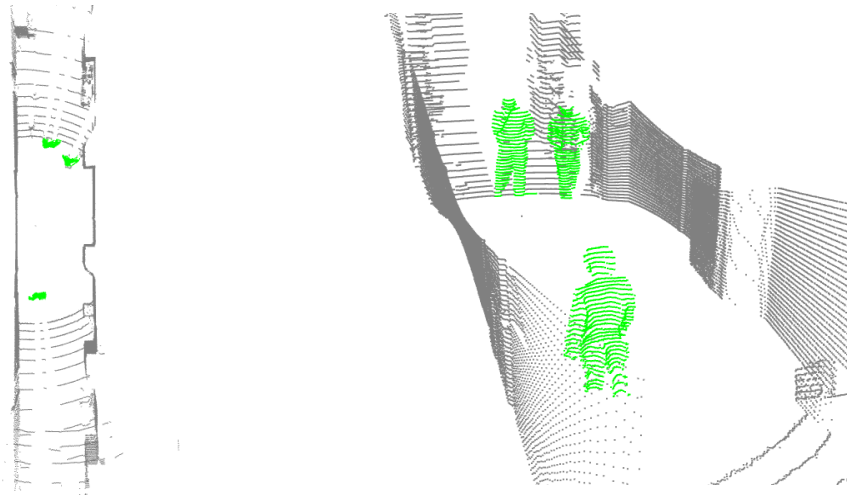
Next, we convert the same frame into a greyscale BEV image and send it to Roboflow's API. Figure 4.3 displays the BEV before and after drawing green bounding boxes around each detected pedestrian.



(a) Greyscale BEV image before detection. *(b) BEV image with predicted bounding boxes.*

Figure 4.3: BEV image generation before and after applying the detector.

Finally, we project those 2D bounding boxes onto the 3D cloud and extract the corresponding points. Figure *4.4* displays the static map (grey) with the extracted pedestrian clusters (green).



(a) Top view of the static map with grey for structures and green for detected pedestrians

(b) Oblique view of the same scene showing the extracted pedestrian clusters.

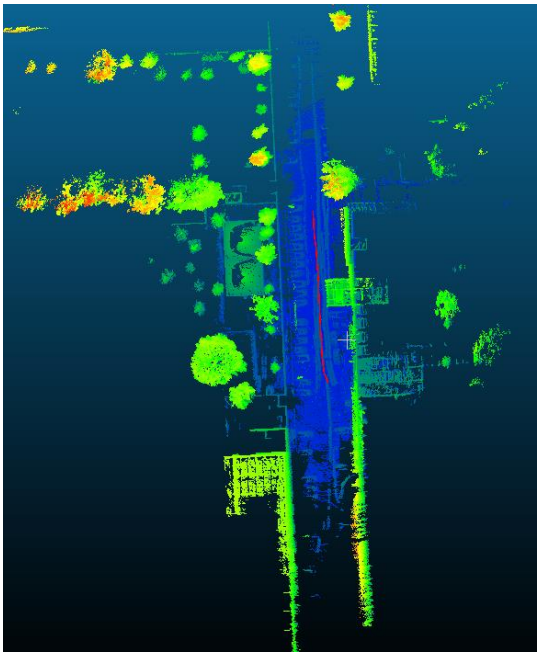
Figure 4.4: Detected pedestrians highlighted on the static point cloud.

Overall, these visuals demonstrate that each BEV detection precisely maps to its corresponding 3D cluster, allowing effective removal of dynamic elements before alignment.

4.2.2. Single-LiDAR SLAM Map Components

After object detection and removal, we evaluate the single-LiDAR SLAM pipeline on Test 3 by generating three complementary maps: a static map, a pedestrian cluster map and the robot's trajectory, all based on the first 400 frames recorded on the street outside the DEIB building.

Figure 4.5 shows the static map in BEV displayed next to the corresponding Google Maps satellite image of the same street.



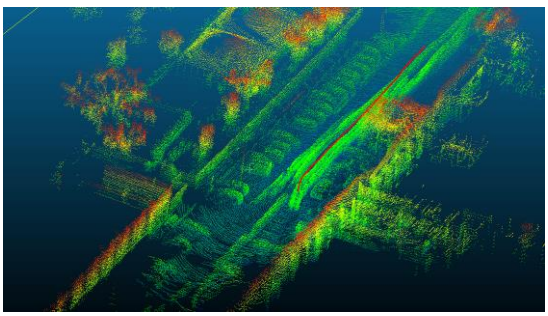
(a) Top view of the static map, showing the street next to the DEIB building with the robot's trajectory in red.

(b) Google Maps satellite view of the same street for comparison.

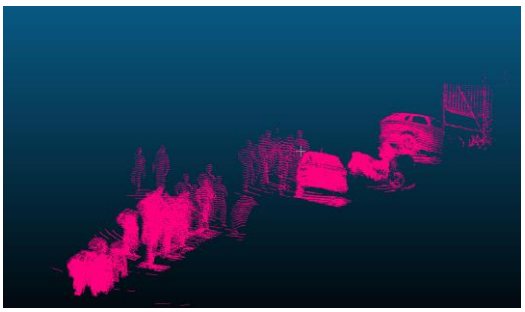
Figure 4.5: Single-robot SLAM results for Test 3 using the first 400 frames.

The reconstructed map closely matches the satellite image, with buildings and trees in place and the robot's path clearly visible.

Figure 4.6(a) presents an oblique view of the static map in a different colour style, showing the parked cars on the street. The green trail next to the red trajectory represents the operators walking behind the robot. Their points were not detected as pedestrians, so they appear as noise. Figure 4.6(b) shows the pedestrian cluster map with all the detections, including some vehicles that were falsely identified as pedestrians.



(a) Oblique view of the static map, showing the street next to the DEIB building with the robot's trajectory in red.



(b) Person map with detected pedestrians in magenta, including some false positives.

Figure 4.6: Static map and pedestrian detections for Test 3.

Finally, Figure 4.7 shows the static map after 550 frames. During the robot's fast turns, ICP fails to fully correct motion distortion, causing the map to drift and appear rotated.

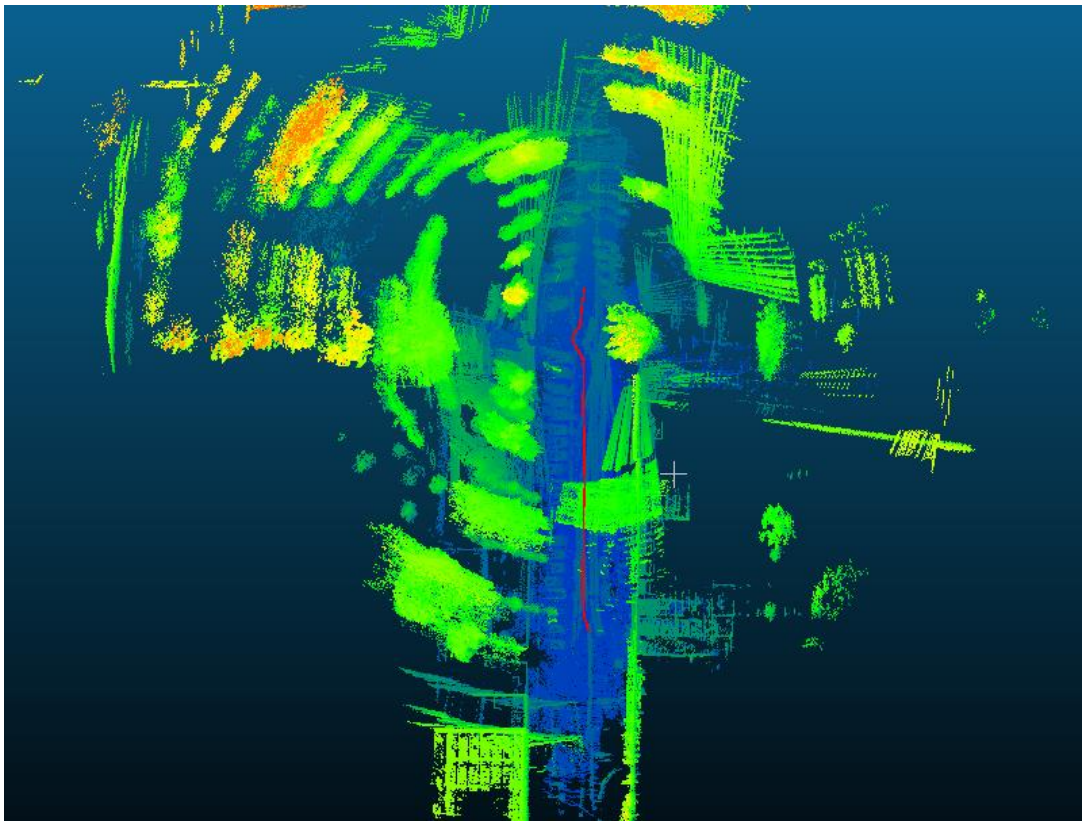


Figure 4.7: Top view of the single-robot SLAM map for Test 3 after 550 frames.

Overall, the static map correctly reconstructs the captured view of the street, the pedestrian clusters show moving elements, and the trajectory verifies accurate pose estimation.

More detailed images, including the other three tests, are provided in Appendix A.

4.2.3. Frame-by-Frame Metrics

To assess the stability and efficiency of the SLAM pipeline, we plot key metrics over the entire run.

First, Figure 4.8 presents the inlier RMSE (left axis) and fitness score (right axis) for each frame in Test 3 (1,737 frames).

Figure 4.8: Inlier RMSE and fitness score per frame for Test 3 (1,737 frames).

The RMSE remains between 0.1 m and 0.14 m for most frames, rising only during the robot's fast turns. Similarly, the fitness score stays above 0.90, dropping in those same segments. This indicates that ICP converges reliably under normal motion but degrades during fast movements. On a laptop with an Intel Core i7-8750 CPU @ 2.20 GHz and 16 GB of RAM (CPU only), the average processing time per frame is 5.57 s, showing that the current implementation cannot run in real time at the LiDAR's 10 Hz capture rate.

Next, Figure 4.9 shows four curves on a single graph: the total number of points in each raw scan, the number of points removed as pedestrians, the number of new points added to the static map after filtering, and the total number of points accumulated in the static cloud.

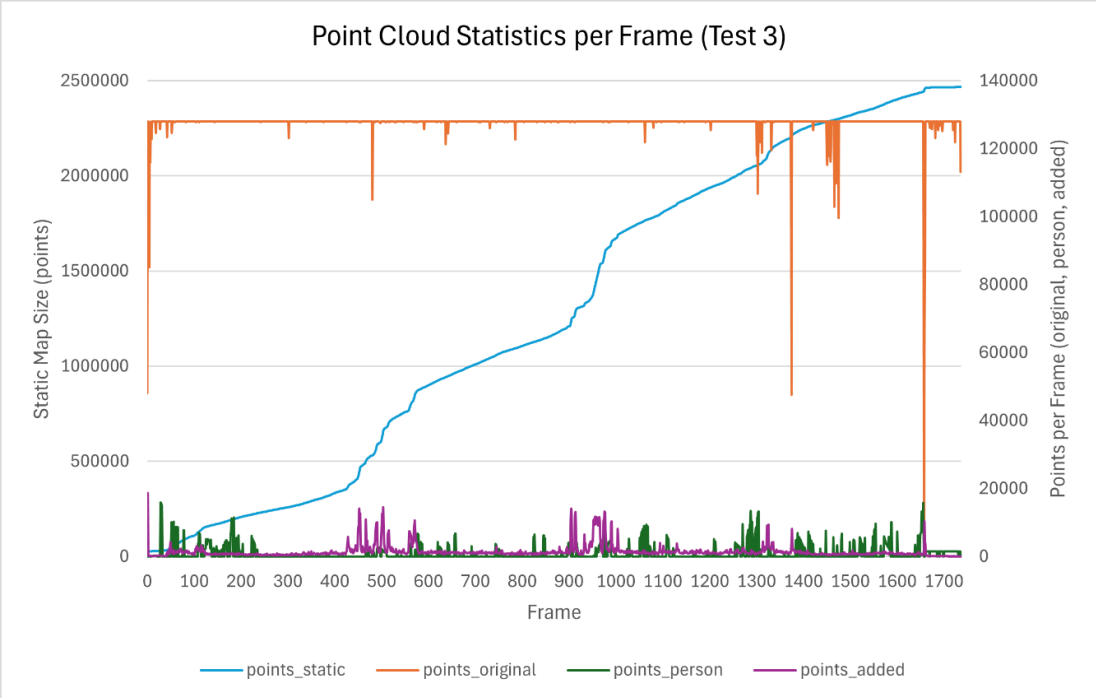


Figure 4.9: Point cloud statistics per frame for Test 3 (1,737 frames).

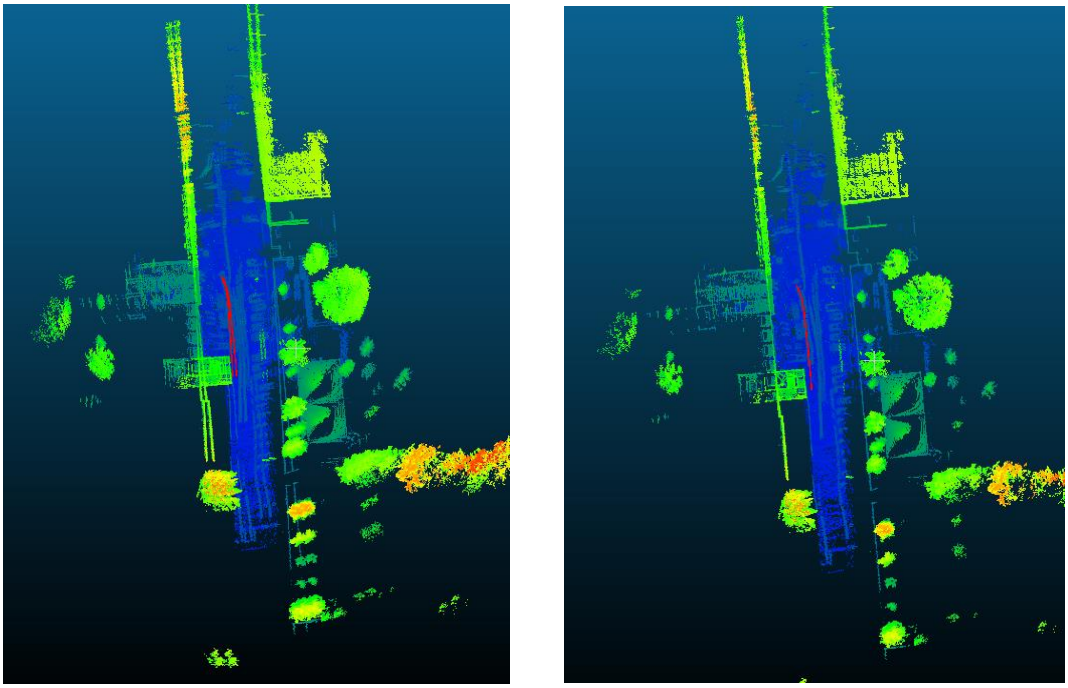
From these curves, we see that only a small number of points are added to the static map, since duplicate filtering removes most points. The static map grows steadily but shows spikes during fast turns, when ICP misalignment fails to detect duplicates and adds extra points.

Overall, RMSE and fitness scores remain stable during normal motion, and the point cloud statistics show effective removal of duplicates and controlled map growth.

Graphics for the other three tests are provided in Appendix A.

4.2.4. Dual-LiDAR Fusion Map

Finally, we present the fused static map obtained by combining the two LiDAR captures. Figure 4.10 shows the fused map of the first 250 frames for Test 3 after aligning the second sensor to the first using a pedestrian as an anchor, performing a rotational search and refining with point-to-point ICP. Figure 4.10(a) shows the static fused map for Test 3, and Figure 4.10(b) the original map for comparison.



(a) Fused static map after aligning and merging the original and rotated captures.

(b) Original single-sensor static map for comparison.

Figure 4.10: Comparison of the first 250 frames of the original and fused static maps for Test 3.

The two maps closely match, with some misalignment visible due to the previously described errors. Comparisons for the other three tests are provided in Appendix A.

4.3. Limitations

Our system relies on the YOLOv8 model to detect the pedestrians, but it frequently misses them or mistakes parts of parked cars for people. Because it was trained only on pedestrians, it cannot detect cars or remove moving vehicles. These errors leave unwanted points in the static map or create false clusters.

The SLAM alignment performs well at a steady pace, but struggles when the robot makes quick turns, causing small drifts that accumulate over time. Access to IMU data could have helped correct this distortion, but it was unavailable for this project.

Finally, our system was never tested with two truly independent LiDAR recordings, where one robot follows another in real time. Setting up such an experiment would also have required training the YOLO model to recognise the robots themselves.

CHAPTER 5

Sustainability Analysis and Ethical Implications

This chapter evaluates the sustainability and ethical implications of the cooperative LiDAR sensing project. First, it introduces a sustainability matrix that summarises the environmental, economic, and social aspects during development, execution, and associated risks. Each perspective is examined in detail. Then, the project's ethical considerations are discussed, and finally we link it to the relevant United Nations Sustainable Development Goals.

5.1. Sustainability Matrix

Table 5.1 below shows the sustainability aspects for the project's development, execution, and key risks and limitations.

	Development of the Bachelor's Project	**Project Execution**	**Risks and Limitations**
Environmental Perspective	Environmental impact	Environmental impact	Environmental risks and limitations
Economic Perspective	Cost	Viability analysis	Economic risks and limitations
Social Perspective	Personal impact	Social impact	Social risks and limitations

Table 5.1: Sustainability matrix.

Below, we will analyse each row in detail.

5.1.1. Environmental Impact

In this section, we estimate the carbon footprint of this project, covering the technology used, transportation, and communication. Most of the work was done

at home on a laptop with an Intel i7-8750 and 16 GB RAM, with some trips to the DEIB lab or the Polimi library. The emissions have been broken down into emails, video calls, laptop power use, LiDAR power consumption, Roboflow API calls, and metro travel.

Email emissions

During the thesis work, 60 emails were exchanged with the Polimi tutor and 46 with the UPC tutor, for a total of 106 emails, of which 11 included attachments. Emissions are estimated as 0.3 gCO_2eq per email without attachments and 50 gCO_2eq per email with attachments [35].

Therefore, the total emissions are:

Email type	Quantity	Emissions per email (gCO_2eq)	Total emissions (gCO_2eq)
Without attachments	95	0.3	28.5
With attachments	11	50	550

Table 5.2: Estimated CO2eq emissions from email exchanges.

In total, emails account for approximately 578.5 gCO2eq.

Video call emissions

During this project, only four meetings were held online (three on Google Meet and one on Microsoft Teams). Each meeting had two participants and lasted about 30 minutes. Emissions are estimated at 0.164 gCO_2eq/minute per person [35].

Number of meetings	Participants	Duration (min)	Emission factor (gCO_2eq/person per minute)	Total emissions (gCO_2eq)
4	2	30	0.164	39.4

Table 5.3: Estimated CO_2eq emissions from the four online meetings.

In total, video calls contributed to approximately 39.4 gCO2eq.

Laptop emissions

For development, a 65 W laptop was used for about 30 hours a week over 18 weeks (540 hours in total).

That's 65 W × 540 h = 35.1 kWh of electricity. With an emission factor of 0.256 $kgCO_2eq/kWh$ [36], this adds up to 9 $kgCO_2eq$ from laptop use.

LiDAR sensor emissions

For capturing the LiDAR recordings, a Hesai PandarXT-32 LiDAR sensor was used, consuming about 15 W [7], for 4 hours.

That's 15 W × 4 h = 60 Wh, or 0.06 kWh. At 0.256 $kgCO_2eq/kWh$, this results in 0.06 kWh × 0.256 $kgCO_2eq/kWh$ = 15.4 gCO_2eq.

Roboflow API call emissions

Approximately 20,000 (around 32 kB each) images were sent to Roboflow, for a total of 640 MB (0.64 GB). At 0.06 kWh/GB [36], this corresponds to 0.64 GB × 0.06 kWh/GB = 0.0384 kWh. Multiplying by 0.256 $kgCO_2eq$ gives 9.8 gCO_2eq from API calls.

Transportation emissions

Twenty round trips were made between home and the DEIB lab and the Polimi library using the Milan metro, each trip being approximately 6.3 km. With emissions of 20 gCO_2eq per passenger per kilometre [37].

That's 6.3 km × 40 trips × 20 gCO_2eq/km = 5,040 gCO_2eq for transportation.

Total emissions

After adding all the sources, the total emissions are:

Source	Emissions (gCO2eq)
Emails	578.5
Video calls	39.4
Laptop use	9,000
LiDAR sensor usage	15.4
Roboflow API calls	10
Metro travel	5,040
Total	14,683.3

Table 5.4: Total estimated CO_2eq emissions from all sources.

This is about 14.7 $kgCO_2eq$ in total. On average, one tree absorbs around 25 $kgCO_2eq$ per year (around 0.5 $kgCO_2eq$/week). Over 18 weeks, that's approximately 9 $kgCO_2eq$, so we would need two trees exclusively to compensate for this thesis's emissions.

5.1.2. Economic Impact

During development, the project required approximately 30 hours of student work per week over 18 weeks (540 hours). Valuing this time at 10 € per hour gives a cost of 5,400 €. A Lenovo IdeaPad laptop (750 €) was used, and all software (VSCode, GitHub Student, Notion, PandarView 2, CloudCompare, Roboflow, common libraries) was free under student or open-source licenses.

For a potential deployment, equipment expenses include two Hesai PandarXT-32 LiDAR sensors (15,000 € each) and two Intel NUC onboard computers (850 € each). Before installation, a viability study is estimated at 40 hours of analysis at 10 €/h. Sensor and computer installation would require 20 hours of technician labour at 15 €/h, plus 100 € for cables, connectors and other materials.

Ongoing maintenance is estimated at 20 hours of technician time per year at 20 €/h, in addition to minimal electricity costs for powering the sensors and computer.

Item	Quantity / Hours	Unit Cost	Total Cost
Development costs			
Student labour	540 h	10 €/h	5,400 €
Lenovo IdeaPad (laptop)	1	750 €	750 €
Software licences	–	–	0 €
Preparation and installation			
Viability study	40 h	10 €/h	400 €
Technician installation	20 h	15 €/h	300 €
Installation materials	–	–	100 €

Deployment equipment			
Hesai PandarXT-32 LiDAR sensor	2	15,000 €	30,000 €
Intel NUC onboard computer	2	850 €	1,700 €
Annual maintenance			
Technician time	20 h/year	20 €/h	400 €/year

Table 5.5: Estimated development, deployment and annual maintenance costs.

The total initial cost is 38,650 €, with a yearly maintenance cost of 400 €.

5.1.3. Social Impact

During development, work was carried out mostly independently under the guidance of two tutors (UPC and Polimi), a lab supervisor, and a colleague who trained the YOLO model. The experience provided hands-on practice with robotics, data processing, and basic AI. Our small team included only one woman (the UPC tutor), so we lacked gender balance.

In operation, the cooperative LiDAR setup could significantly improve road safety by sharing sensor data between vehicles, leading to autonomous driving in the future. While this automation might reduce jobs such as taxi driving, it could also offer safer, independent transport for elderly or disabled people. To protect privacy, pedestrian detection must be accurate enough that any point cloud containing a person can be anonymised or deleted.

There are a few important risks and limitations. Because our model was trained on just a few men of average height, it may underperform for women, children, or people with different body types. Detection errors could also lead to accidents if strict safety protocols are not in place. Finally, the lack of female representation among the test subjects shows the need to involve more diverse participants in the future.

5.2. Ethical Implications

This project aims to improve road safety by allowing vehicles to share LiDAR data, but it also raises important ethical questions. First, there is a clear benefit: detecting pedestrians, cyclists, and obstacles could reduce accidents and save lives. However, there are unintended consequences. If data-sharing policies are not carefully designed, this information can be used for surveillance or tracking individuals. Since the model was trained on only a few men, there is also the risk of bias, and it may underperform for women or children, leading to dangerous situations.

To address these issues, any continuation of this work should adhere to established standards such as the UPC Code of Research Integrity, ensuring transparency, data minimisation, and informed consent. Point clouds that contain identifiable people should be deleted or anonymised, and future tests should include participants of different genders, ages, and body types so that the system treats all road users fairly.

5.3. Relationship with the Sustainable Development Goals

This project directly contributes to some Sustainable Development Goals (SDGs) adopted by the United Nations:

SDG 3: Good Health and Well-being

By improving detection of vulnerable road users (e.g., pedestrians, cyclists), the cooperative LiDAR sensing system helps prevent accidents and reduce injuries and fatalities on roads.

SDG 9: Industry, Innovation and Infrastructure

By allowing vehicles to share sensor data in real time, this work supports smarter transport systems and opens the door to more efficient technology driven mobility solutions.

SDG 11: Sustainable Cities and Communities

Reducing road accidents contributes to safer, more liveable urban environments. In the future, autonomous driving could offer transport options that are more inclusive for all residents.

CHAPTER 6

Conclusions and Future Work

In this final chapter, we summarise the results of this thesis, address how well they meet the initial objectives, and then outline steps for future development.

6.1. Conclusions

This thesis has built and tested a full pipeline for processing LiDAR point clouds in vehicular scenarios, with the goal of improving mapping accuracy and situational awareness.

For the single-LiDAR SLAM pipeline, combining point-to-point ICP with statistical filtering and a YOLOv8 pedestrian detector produced a clean static map that closely matches the real environment. We achieved an RMSE below 0.14 m and a fitness score above 0.90 during normal robot motion. The robot's path was recovered accurately, and redundant points were successfully removed to keep the map size under control.

In the dual-LiDAR simulation, we used a shared pedestrian as an anchor to align two rotated copies of the same dataset. After a translation, a rotation search, and a final ICP refinement, we obtained an RMSE below 0.10 m. Although this was a simulated scenario, it shows that fusion based on object detection can produce reliable alignments even when there is little overlap between scans, as long as the detector works well.

All original objectives have been met:

1. Each new scan is registered to the robot's growing local map.
2. The robot's trajectory is reconstructed from the accumulated transformations.
3. A YOLO-based model detects pedestrians in the raw point clouds.
4. Detected pedestrians are removed, keeping only static elements.
5. Pedestrians are used as anchors to estimate the relative pose between the two robots.

6. After rotation search and an ICP refining, both maps fuse into a single, more complete view.

By combining open-source Python tools with the Hesai PandarXT-32 LiDAR sensor, this thesis contributes to the development of a flexible framework for 3D mapping, merging multiple LiDAR views into a single, detailed model of the environment.

6.2. Future Directions

This thesis lays the groundwork for a cooperative LiDAR mapping system, but significant improvements are needed before a real-world deployment.

First, the current code is split into two main scripts. Restructuring it into a modular framework such as a ROS (Robot Operating System) package would make the system easier to maintain, extend, and integrate with larger robotic platforms.

Second, our SLAM pipeline struggles during fast turns. Integrating IMU data or applying motion models, such as estimating the robot's velocity between scans, would help correct motion distortion and reduce drift. Exploring alternative alignment methods such as GICP or NDT could also improve robustness.

Third, the YOLO model must be retrained on a larger dataset that includes vehicles, the robots themselves, and people from different perspectives. This will reduce false positives and make the detection anchors more reliable. Adding temporal tracking to link detections across consecutive frames will improve object detection and map fusion, since each frame is currently processed independently.

Finally, we need to test with two real LiDAR sensors working together. Recording two robots moving in sync will show real-world challenges such as objects blocking the view or small timing errors. If each robot detects three or more common objects, the maps could fuse almost instantly. Alternatively, the vehicles could share their orientation or GPS data to align their maps without requiring a 360° search.

Addressing these points will move the system towards real-time operation and deployment in complex, dynamic environments, and enable safer, smarter cooperation between vehicles.

Bibliography

[1] P. J. Besl and N. D. McKay, "A method for registration of 3-D shapes," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 14, no. 2, pp. 239-256, Feb. 1992, doi: 10.1109/34.121791.

[2] C. Cadena et al., "Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age," in IEEE Transactions on Robotics, vol. 32, no. 6, pp. 1309-1332, Dec. 2016, doi: 10.1109/TRO.2016.2624754.

[3] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 779-788, doi: 10.1109/CVPR.2016.91.

[4] Hesai Technology, PandarView 2: Point Cloud Visualization Software User Manual, Doc Version: PV2-en-230810, internal documentation, 2021.

[5] Open3D Documentation. "ICP registration." Open3D, Version 0.19.0. [Online]. Available: https://www.open3d.org/docs/release/tutorial/pipelines/icp_registration.html. [Accessed: 12 March 2025].

[6] Daniel Girardeau-Montaut. CloudCompare - 3D point cloud and mesh processing software, GPL software, Open Source Project. [Online]. Available: https://www.cloudcompare.org/.

[7] Hesai Technology, PandarXT-32: 32-Channel Medium-Range Mechanical LiDAR User Manual, Doc Version: X01-en-2002A2, internal documentation, 2020.

[8] Roboflow. "Train YOLOv8 on a Custom Dataset." [Online]. Available: https://docs.roboflow.com/train/train. [Accessed: 27 February 2025].

[9] J. L. Bentley. "Multidimensional binary search trees used for associative searching." Communications of the ACM, vol. 18, no. 9, pp. 509–517, 1975. DOI: 10.1145/361002.361007.

[10] Vizzo, Ignacio & Guadagnino, Tiziano & Mersch, Benedikt & Wiesmann, Louis & Behley, Jens & Stachniss, Cyrill. (2023). KISS-ICP: In Defense of Point-to-Point ICP Simple, Accurate, and Robust Registration If Done the Right Way. IEEE Robotics and Automation Letters. PP. 1-8. 10.1109/LRA.2023.3236571.

[11] J Ma, S L Zhuo, L Qiu, Y Z Gao, Y F Wu, M Zhong, R Bai, M Sun, and P Y Chiang, A review of ToF-based LiDAR[J]. J. Semicond., 2024, 45(10), 101201 doi: 10.1088/1674-4926/24040015.

[12] A. Martinelli, "Vision and IMU Data Fusion: Closed-Form Solutions for Attitude, Speed, Absolute Scale, and Bias Determination," in IEEE Transactions on Robotics, vol. 28, no. 1, pp. 44-60, Feb. 2012, doi: 10.1109/TRO.2011.2160468.

[13] Point Cloud Library. "Statistical Outlier Removal filter tutorial." [Online]. Available: https://pcl.readthedocs.io/projects/tutorials/en/latest/statistical_outlier.html. [Accessed: 24 March 2025].

[14] Open3D Documentation. “Point-cloud outlier removal.” [Online]. Available: https://www.open3d.org/docs/latest/tutorial/Advanced/pointcloud_outlier_removal.html. [Accessed: 17 June 2025].

[15] Point Cloud Library. “VoxelGrid filter tutorial.” [Online]. Available: https://pointclouds.org/documentation/tutorials/voxel_grid.html. [Accessed: 20 March 2025].

[16] Ding, Zihuan. (2023). “3D registration process of ICP using linear constraint”. Advances in Engineering Technology Research. 4. 542. 10.56028/aetr.4.1.542.2023.

[17] P. Li, R. Wang, Y. Wang and W. Tao, "Evaluation of the ICP Algorithm in 3D Point Cloud Registration," in IEEE Access, vol. 8, pp. 68030-68048, 2020, doi: 10.1109/ACCESS.2020.2986470.

[18] Segal, Aleksandr & Hähnel, Dirk & Thrun, Sebastian. (2009). Generalized-ICP. Proc. of Robotics: Science and Systems. 10.15607/RSS.2009.V.021.

[19] P. Biber and W. Strasser, "The normal distributions transform: a new approach to laser scan matching," Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003) (Cat. No.03CH37453), Las Vegas, NV, USA, 2003, pp. 2743-2748 vol.3, doi: 10.1109/IROS.2003.1249285.

[20] R. B. Rusu, N. Blodow and M. Beetz, "Fast Point Feature Histograms (FPFH) for 3D registration," 2009 IEEE International Conference on Robotics and Automation, Kobe, Japan, 2009, pp. 3212-3217, doi: 10.1109/ROBOT.2009.5152473.

[21] Shi, Pengcheng & Yan, Shaocheng & Xiao, Yilin & Liu, Xinyi & Zhang, Yongjun & Li, Jiayuan. (2024). “RANSAC Back to SOTA: A Two-Stage Consensus Filtering for Real-Time 3D Registration”. IEEE Robotics and Automation Letters. PP. 1-8. 10.1109/LRA.2024.3502056.

[22] V. A. Vaduvescu and P. Negrea, "Inertial Measurement Unit – A Short Overview of the Evolving Trend for Miniaturization and Hardware Structures," 2021 International Conference on Applied and Theoretical Electricity (ICATE), Craiova, Romania, 2021, pp. 1-5, doi: 10.1109/ICATE49685.2021.9465024.

[23] S. Bedín, J. Civera and M. Nitsche, "Teach and Repeat and Wheel Calibration for LiDAR-Equipped Omnidirectional Drive Robots," 2023 European Conference on Mobile Robots (ECMR), Coimbra, Portugal, 2023, pp. 1-6, doi: 10.1109/ECMR59166.2023.10256356.

[24] Yan, Y., Mao, Y., & Li, B. (2018). SECOND: Sparsely Embedded Convolutional Detection. Sensors, 18(10), 3337. https://doi.org/10.3390/s18103337.

[25] M. Ye, S. Xu and T. Cao, "HVNet: Hybrid Voxel Network for LiDAR Based 3D Object Detection," in 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Seattle, WA, USA, 2020, pp. 1628-1637, doi: 10.1109/CVPR42600.2020.00170.

[26] Lang, Alex H. et al. “PointPillars: Fast Encoders for Object Detection From Point Clouds.” 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (2018): 12689-12697.

[27] C. Liu, L. Zhou, Y. Huang and A. Knoll, "YOLO-BEV: Generating Bird's-Eye View in the Same Way as 2D Object Detection," arXiv preprint arXiv:2310.17379, 2023.

[28] Fan, Lue & Xiong, Xuan & Wang, Feng & Wang, Naiyan & Zhang, Zhaoxiang. (2021). RangeDet: In Defense of Range View for LiDAR-based 3D Object Detection. 2898-2907. 10.1109/ICCV48922.2021.00291.

[29] Rohit Kundu, "YOLO: Algorithm for Object Detection Explained [+Examples]," V7 Labs blog, Jan. 17, 2023. [Online]. Available: https://www.v7labs.com/blog/yolo-object-detection. [Accessed: 8 April 2025].

[30] Ultralytics. "One-Stage Object Detectors Explained," Ultralytics Glossary. [Online]. Available: https://www.ultralytics.com/glossary/one-stage-object-detectors. [Accessed: 14 April 2025].

[31] Wei, Pengjin & Yan, Guohang & Li, Yikang & Fang, Kun & Liu, Wei & Cai, Xinyu & Yang, Jie. (2022). CROON: Automatic Multi-LiDAR Calibration and Refinement Method in Road Scene. 10.48550/arXiv.2203.03182.

[32] Thomas, Aby & Sunilkumar, Adarsh & Shylesh, Shankar & T., Aby & Methirumangalath, Subhasree & Dong, Chen & Peethambaran, Jiju. (2020). TCM-ICP: Transformation Compatibility Measure for Registering Multiple LIDAR Scans. 10.48550/arXiv.2001.01129.

[33] Zhao, Jingwen & Huang, Zhicong & Zheng, Zhijie & Long, Yunliang & Hu, Haifeng. (2025). Semantic-Guided Multi-Feature Attention Aggregation Network for LiDAR-Based 3D Object Detection. Electronics. 14. 2154. 10.3390/electronics14112154.

[34] A. Lathwal, "Point Cloud Downsampling Methods and Python Implementations," Medium, 2023. [Online]. Available: https://medium.com/@lathwath5/point-cloud-downsampling-methods-and-python-implementations-6f91a129ac48. [Accessed: 23 April 2025].

[35] Carbon Literacy Trust, "The Carbon Cost of an Email," Carbon Literacy Project, 2022. [Online]. Available: https://carbonliteracy.com/the-carbon-cost-of-an-email/. [Accessed: 17 June 2025].

[36] J. Aslan, K. Mayers and J. G. Koomey, "Electricity Intensity of Internet Data Transmission: Untangling the Estimates," Journal of Industrial Ecology, 2018. [Online]. Available: https://doi.org/10.1111/jiec.12630. [Accessed: 17 June 2025].

[37] T. Bellanto, "Quanto inquinano i mezzi di trasporto? Un'analisi dell'anidride carbonica emessa da ogni mezzo," Viviamo Sostenibile, Apr. 2021. [Online]. Available: https://viviamosostenibile.it/mezzi-di-trasporto-quali-preferire-per-muoversi-in-maniera-ecologica. [Accessed: 18 June 2025]

APPENDIX A

Detailed SLAM and Fusion Results

In the following pages we show, for Tests 1, 2 and 4, the frame-by-frame metrics, point cloud statistics, single-LiDAR maps and fused maps referred to in Chapter 4.

A.1. Test 1

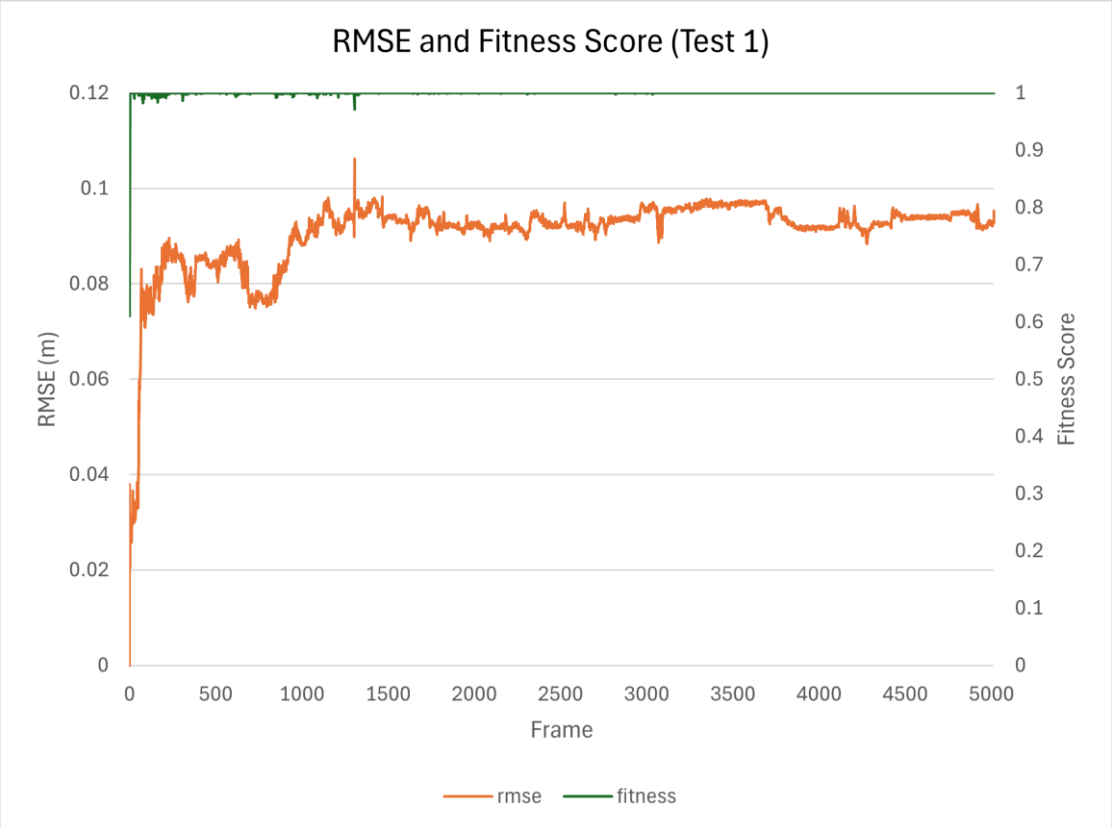


Figure A.1: Inlier RMSE and fitness score per frame for Test 1 (5,016 frames).

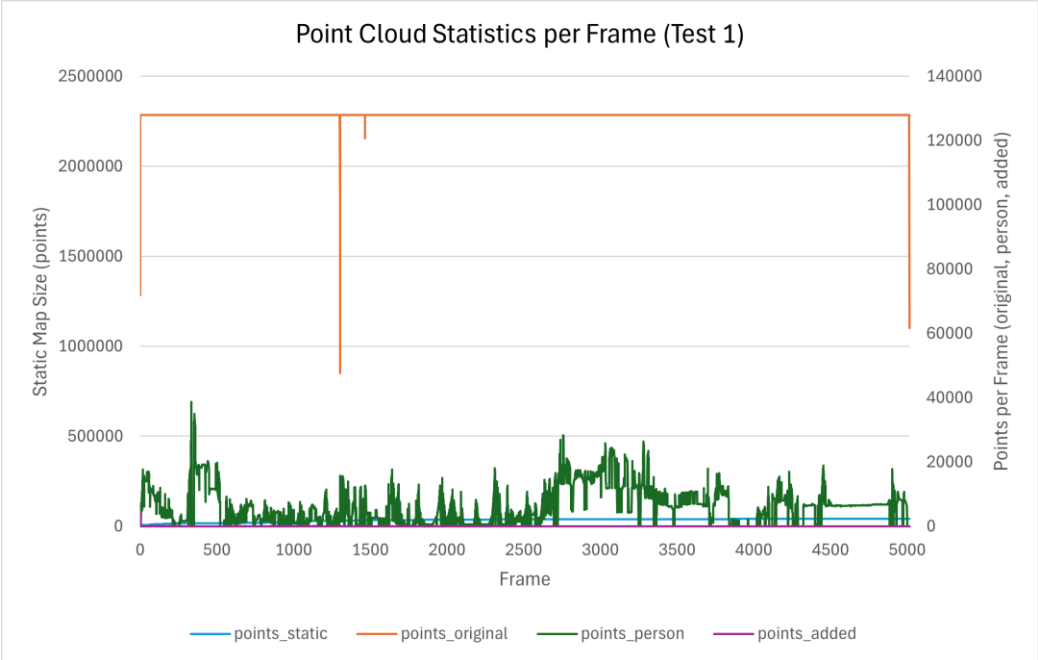
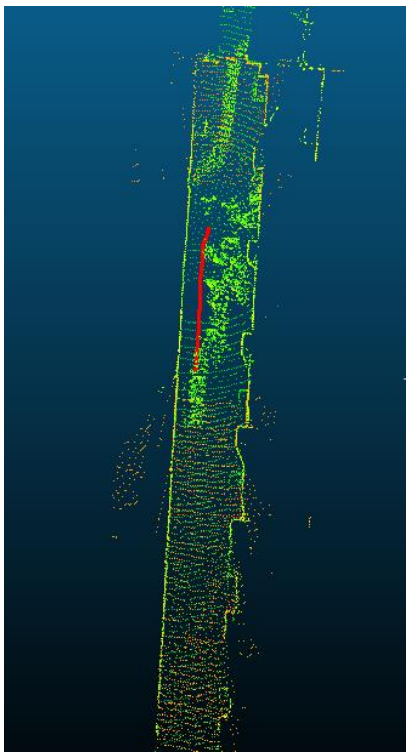
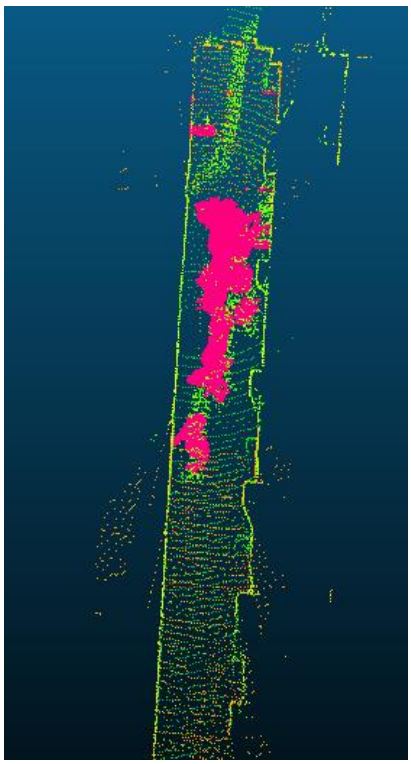


Figure A.2: Point cloud statistics per frame for Test 1 (5,016 frames).

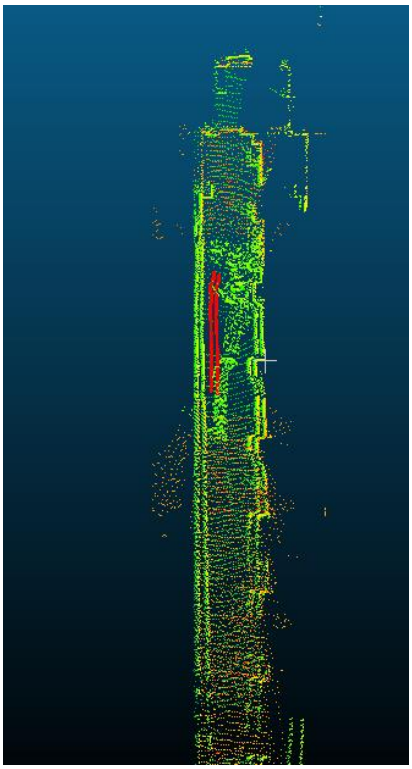


(a) Top view of the static map and trajectory for the first 450 frames of Test 1, with detected pedestrians removed.

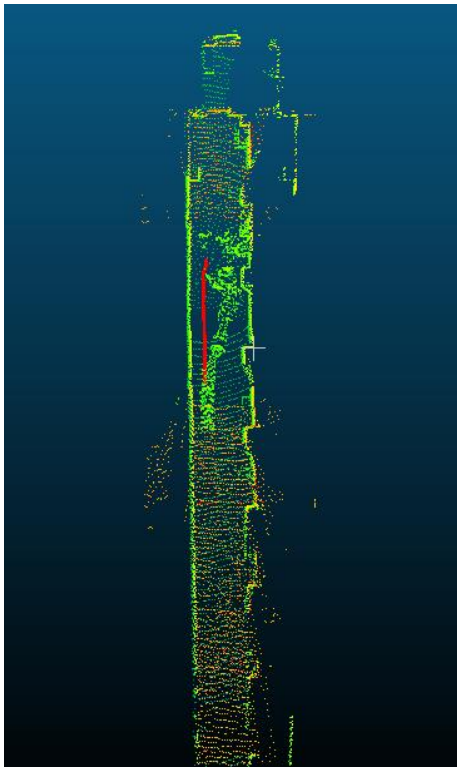


(b) Top view of the same map, showing the removed pedestrian points in magenta.

Figure A.3: Static map and trajectory for Test 1 comparing with and without displayed pedestrian detections.



(a) Fused static map after aligning and merging the original and rotated captures.



(b) Original single-sensor static map for comparison.

Figure A.4: Comparison of fused and single-sensor static maps for Test 1 (first 250 frames).

A.2. Test 2

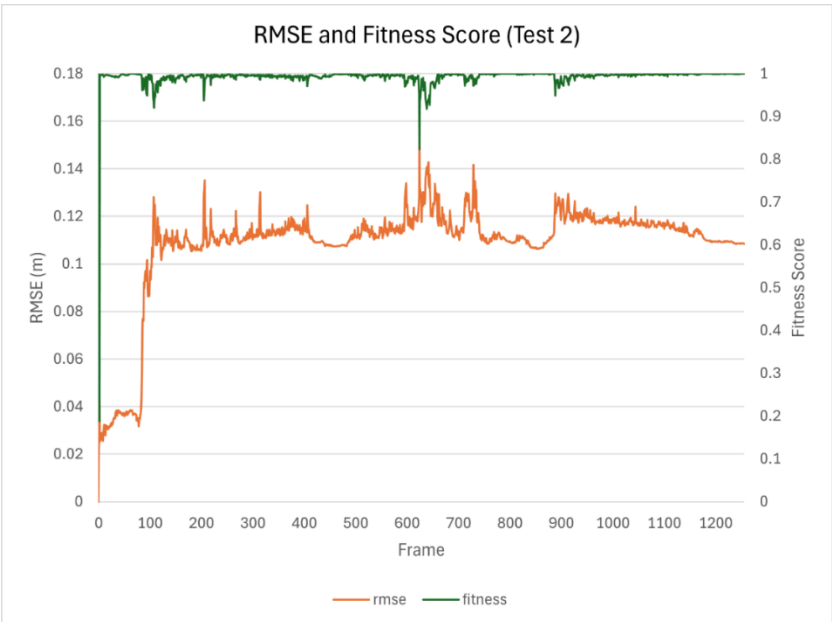


Figure A.5: Inlier RMSE and fitness score per frame for Test 2 (1,256 frames).

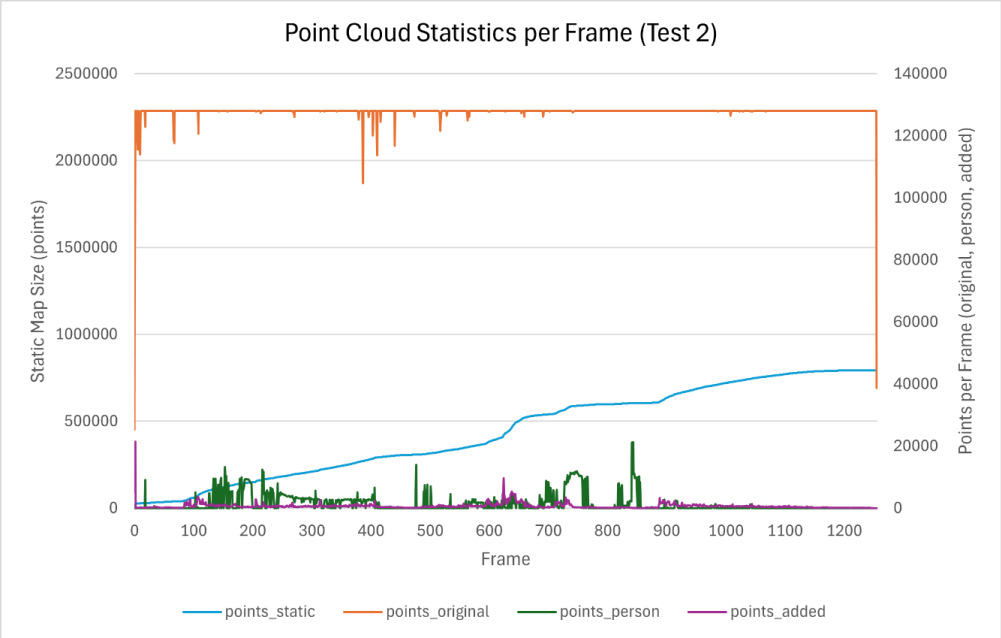
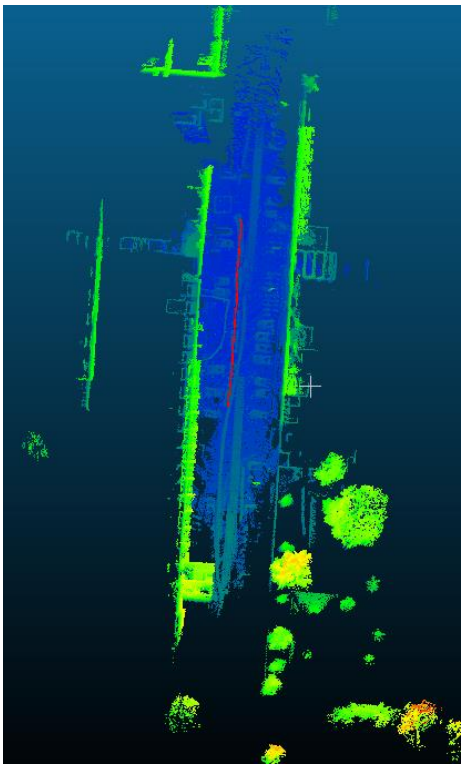
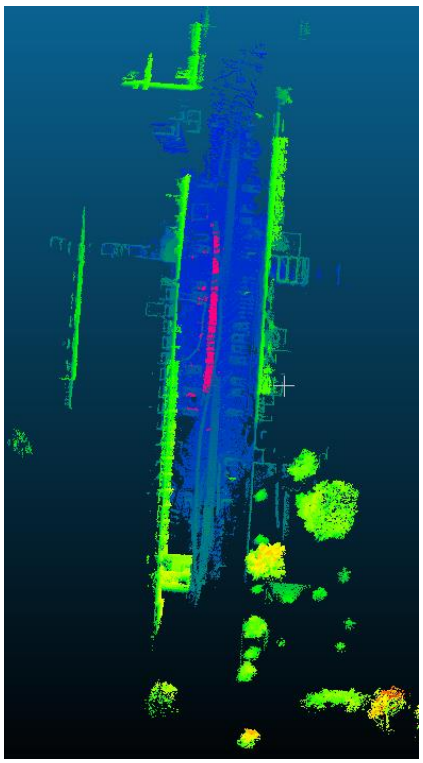


Figure A.6: Point cloud statistics per frame for Test 2 (1,256 frames).

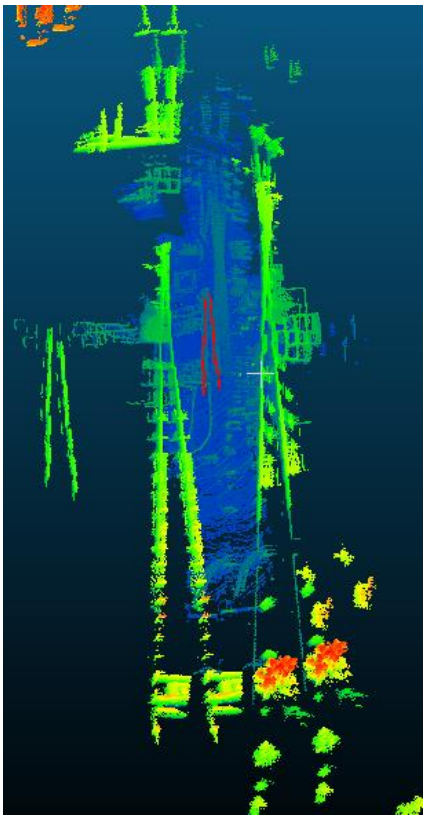


(a) Top view of the static map and trajectory for the first 450 frames of Test 2, with detected pedestrians removed.

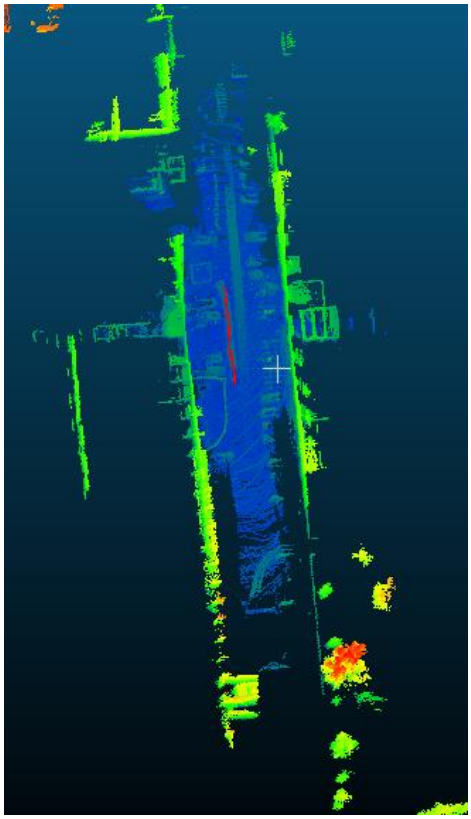


(b) Top view of the same map, showing the removed pedestrian points in magenta.

Figure A.7: Static map and trajectory for Test 2 comparing with and without displayed pedestrian detections.



(a) Fused static map after aligning and merging the original and rotated captures.



(b) Original single-sensor static map for comparison.

Figure A.8: Comparison of fused and single-sensor static maps for Test 2 (first 250 frames).

A.3. Test 4

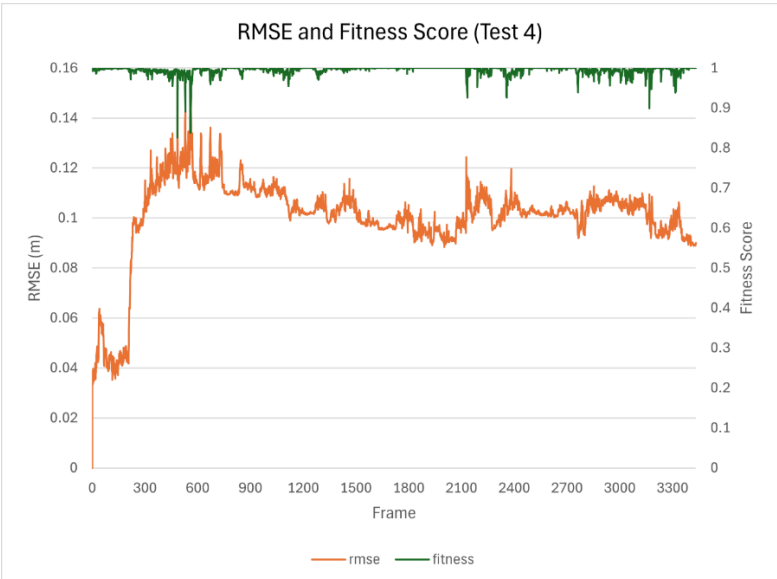


Figure A.9: Inlier RMSE and fitness score per frame for Test 4 (3,434 frames).

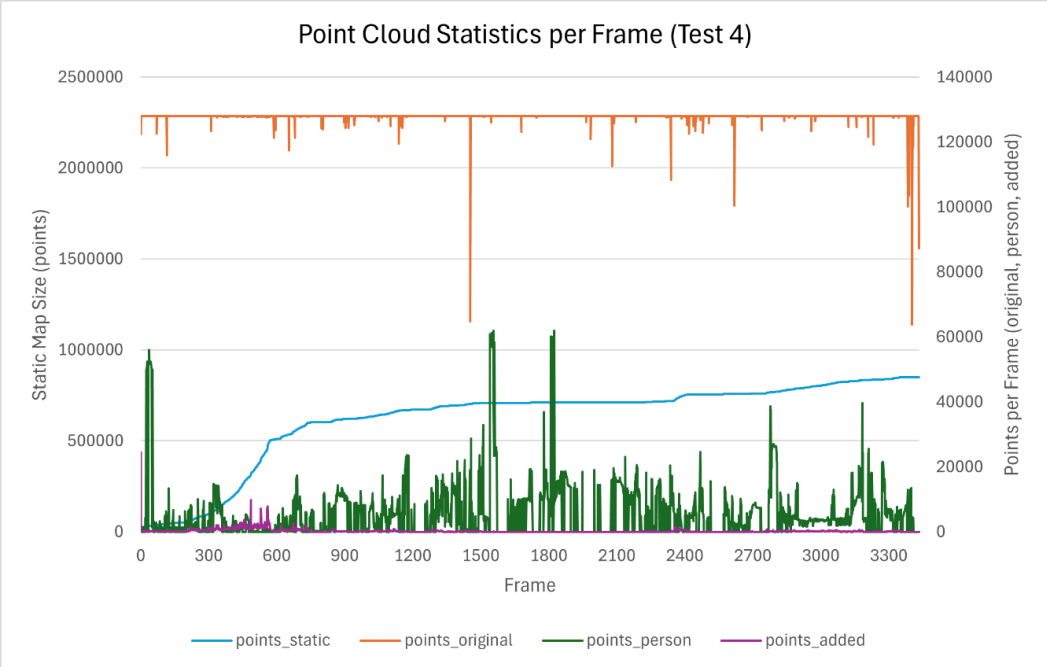
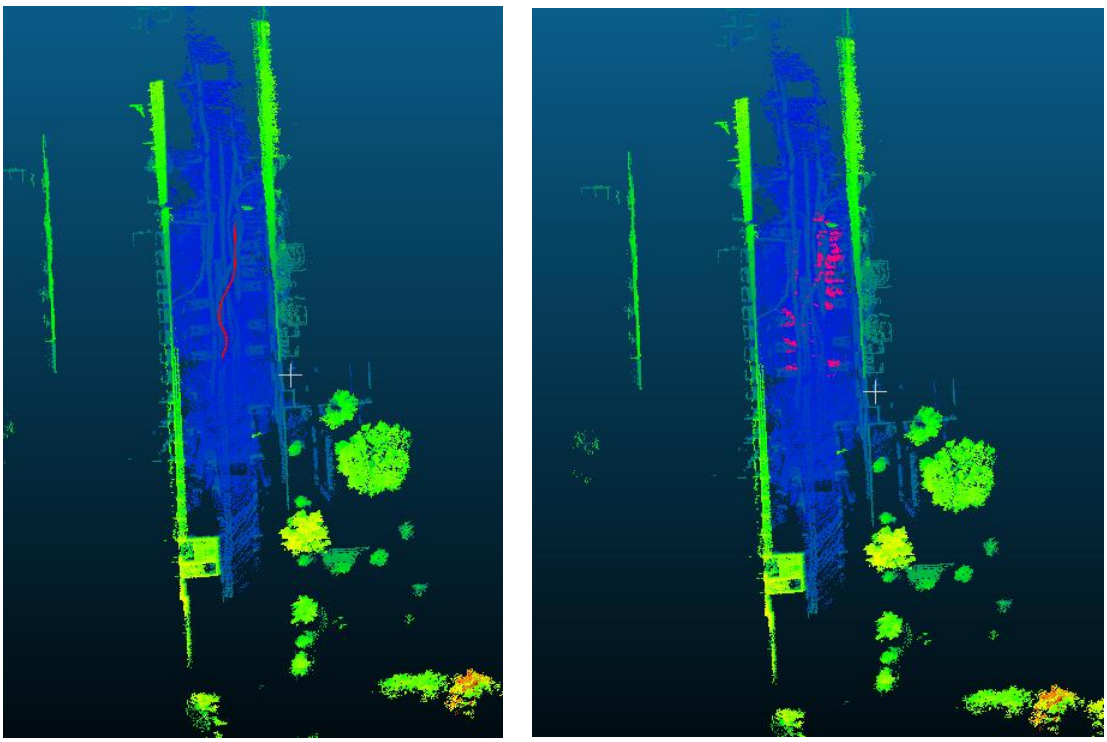


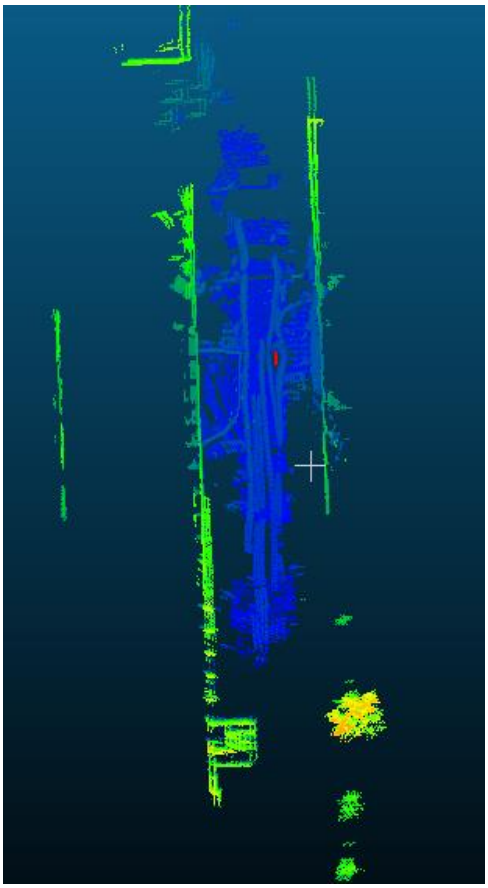
Figure A.10: Point cloud statistics per frame for Test 4 (3,434 frames).



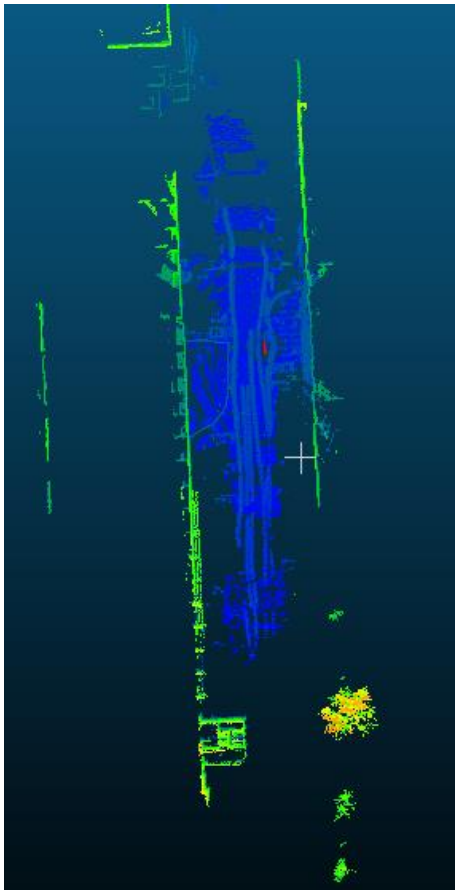
(a) Top view of the static map and trajectory for the first 450 frames of Test 4, with detected pedestrians removed.

(b) Top view of the same map, showing the removed pedestrian points in magenta.

Figure A.11: Static map and trajectory for Test 4 comparing with and without displayed pedestrian detections.



(a) Fused static map after aligning and merging the original and rotated captures.



(b) Original single-sensor static map for comparison.

Figure A.12: Comparison of fused and single-sensor static maps for Test 4 (first 250 frames).